
Agentic PCG: Procedural Content Generation via Tool-using LLMs

Zehua Jiang

Department of Computer Science
New York University
Brooklyn, NY 11201
zehua.jiang@nyu.edu

Sam Earle

Department of Computer Science
New York University
Brooklyn, NY 11201
sam.earle@nyu.edu

Ahmed Khalifa

Institute of Digital Games
University of Malta
Msida, Malta
ahmed@khalifa.com

Julian Togelius

Department of Computer Science
New York University
Brooklyn, NY 11201
julian@togelius.com

Abstract

Procedural Content Generation (PCG) provides a means of automatically generating diverse, complex functional environments for video game playing agents. While various optimization approaches have been used to generate content meeting computable functional constraints, more open-ended human priors and directives are difficult to bake in to such methods. We propose *Agentic Procedural Content Generation*, a paradigm in which tool-calling large language models generate video game levels that are both functional and steerable via natural language. Equipped with local brushes, generative algorithms, and evaluation functions, agents iteratively refine content toward specified design objectives. We show that the framework applies to both static level design tasks and environments with dynamic gameplay mechanics, while satisfying intended functional constraints. Beyond such constraints, the same framework can respond to open-ended gameplay prompts and produce levels with stronger structural and stylistic coherence than purely metric-driven search, suggesting that Agentic PCG is a framework for balancing functional constraint satisfaction with free-form design control.

1 Introduction

The advent of agentic AI implemented as harnesses on large language models has prompted many to rethink their processes. Workflows, habits, and whole applications are reimaged in the form of reasoning LLMs that call tools to probe and poke at the world. How about game content production pipelines?

A large variety of games, including but certainly not limited to the multitude of roguelikes and roguelites, feature procedural generation of some aspect of game content [1]. In most cases, the PCG solution is handcrafted for each game, although algorithmic strategies are reused. There is also a blossoming field of PCG research in academia, where various such algorithmic strategies are developed. These strategies vary widely, but many fall into the wider generate-and-test paradigm, where editing actions are interleaved with automated evaluation of the generated content.

What we propose here is to use the wide variety of methods developed within PCG as building blocks for LLM-driven agentic workflows. This amounts to deconstructing existing PCG methods and reusing both their editing and evaluation parts as tools that can be called by a properly prompted

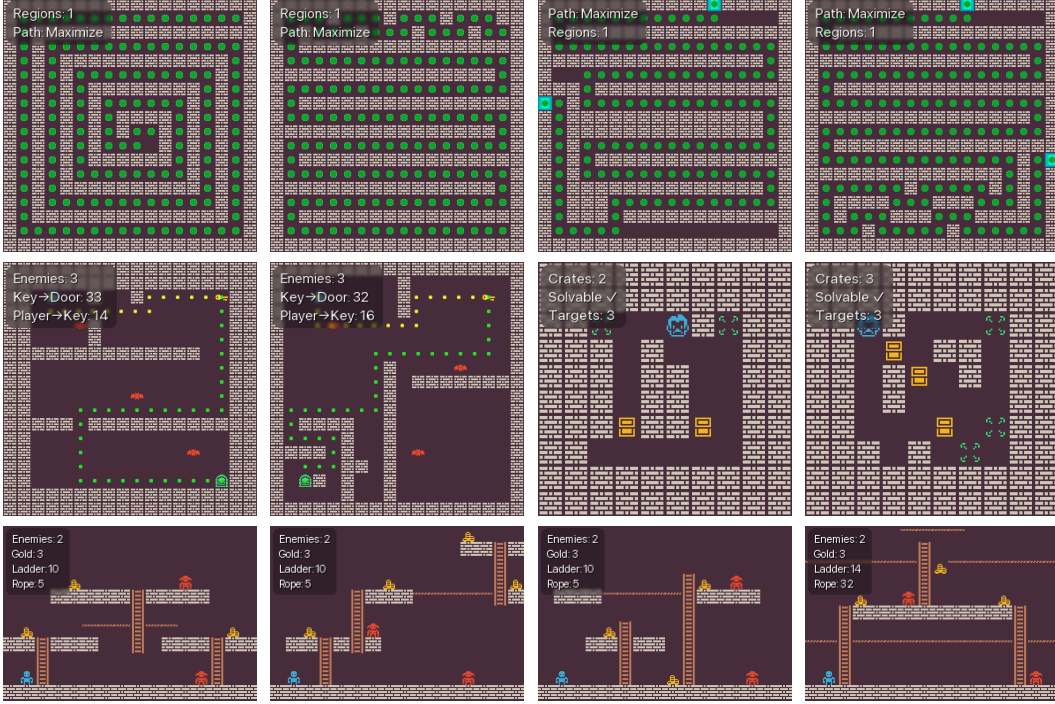


Figure 1: **Overview examples.** **Binary** and **Binary Door** agents discover zig-zag-like long-path structures and adapt them to door constraints (top row). **Zelda** and **Sokoban** examples satisfy object-placement and reachability constraints in compact rooms (middle row). **Lode Runner** examples adjust ladders, ropes, gold, and enemies to satisfy functional constraints (bottom row). Target metrics on the top left tag, respectively.

LLM. The promise of this approach is to make it much easier to create content generators for specific games while reusing existing code. We call this approach Agentic Procedural Content Generation.

The bulk of this paper describes a system in which a powerful LLM is given access to a number of tools for editing and evaluating levels, and given prompts to generate them. To show the versatility of our approach, we generate levels for Binary Maze, Binary Door, Super Mario Bros, Zelda, Lode Runner, and Sokoban. The environments are mainly taken from the PCG benchmark [2], which provides a standard API for evaluation functions.

A key concern for most game developers is that the method runs fast and on device, as the economics of most games do not permit querying frontier models at runtime. To this end, we investigate the capability degradation of Agentic PCG when using local LLMs in the orchestrating role. By grounding agents with domain-specific tools and evaluation metrics, we enable models with varying capacity to iterate productively.

Our key contribution is a unified planning, feedback, and optimization framework for procedural content generation using tool-calling LLMs. First, we show that the harness is effective in both static design settings and environments with dynamic gameplay mechanics. Second, that the framework is not restricted to primitive edit actions such as tile placement, but it can also integrate higher-level PCG algorithms as callable tools for level editing. Third, that it can incorporate open-ended natural language requirements in addition to explicit functional constraints, enabling qualitative steering of broader gameplay qualities alongside measurable design targets.¹

¹Code: <https://github.com/JiangZehua/AgenticPCG> Project Page: <https://zehua-jiang.github.io/AgenticPCG/>

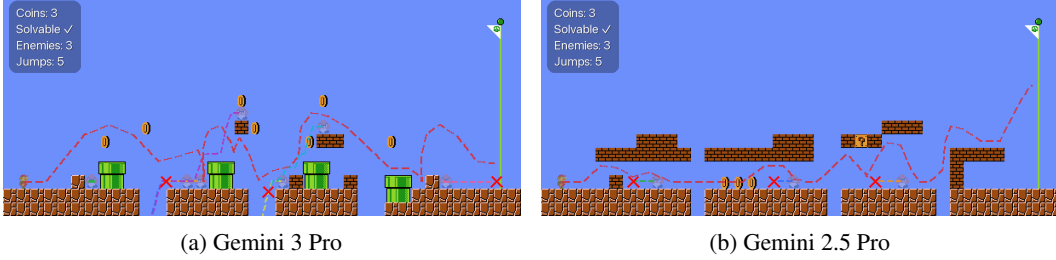


Figure 2: **Super Mario Bros.** The red dashed curve shows the simulated player path; colored trajectories show Goomba motion; transparent sprites mark initial positions; red crosses mark kills. Agents use pits, ceilings, and enemy placement to induce target jumps and kills.

2 Related Work

While many content generators deployed in games use ad hoc constructive approaches, the research field around PCG has mostly focused on approaches based on optimization and/or machine learning.

Search-based procedural content generation (SBPCG) [3] relies on search or optimization algorithms to search the space of possible content with the help of an evaluation function. The evaluation function can be as simple as evaluating statistics about the current content (direct evaluation), or as complex as simulation-based evaluation in which an AI agent uses the content during play (e.g. playing through a level) and evaluation is applied to the resulting playtrace. Search-based PCG has been used to create various types of content, such as 3d models [4], tutorials [5], levels [6], rules [7], and full games [8]. Procedural content generation via machine learning (PCGML) [9] uses input examples instead of an evaluation function to learn how to generate content. Instead of generating the content directly, machine learning is used to train a generator that can produce content that is similar to, yet distinct from, the input examples. Various generative models have been used, including N-Grams [10], Markov Chains [11], LSTMs [12], AutoEncoders [13], and GANs [14].

With the rise of Large Language Models (LLMs), researchers have explored the training and usage of LLMs to generate different types of content, such as game levels [15–17], game maps [18], game mechanics [19], and puzzle games [20]. One of the biggest advantages of using LLMs is the capacity for designers to express specifications for the generated content using natural language, enabling easier and more flexible interaction with the system.

Gallotta et al. [21, 22] investigate the use of natural language within a mixed initiative system [23], probing its ability to act as a conduit for capturing designer intent. Instead of modifying actions by hand, the designer can directly express a high-level plan or design goal in natural language, which an LLM then converts into multiple tool calls (e.g. creating a room, adding enemies). The results show that decomposing large tasks into smaller ones via multiple LLM calls is more effective than asking the system to make large changes in a single pass. Our work expands on Gallotta et al.’s framework by allowing the LLM to be an agentic system in which the LLM can not only call tools to directly edit the content, but also nested PCG functions. We also explore the LLM as a fully autonomous system as opposed to a mixed initiative system.

Even with recent advances, LLMs are still prone to hallucinations and mistakes. Generating functional content such as game levels (that not only need to look good but also be playable and solvable) is a very hard task for LLMs as they cannot easily capture the spatial connections between different game tiles in the level. Unaided LLMs also often have problems correctly counting the number of objects in the level, or finding paths between objects. Most crucially, they cannot simulate the game to make sure that it is playable. These problems call for using external tools, both to inform the LLM and to allow it to affect the right kind of changes to the level.

A major inspiration for this work is the PCGRL line of research [24–28], which formulates level editing as a reinforcement learning problem: an agent traverses the level, selects local edit actions at each position, and learns a controllable and robust policy from rewards defined by level evaluation metrics. Similarly, we cast level design as an iterative, RL-like task, with a reward function defined by distance from a set of target values over a number of evaluation metrics. The agent’s “action space”, however, is much vaster in our work, in which LLMs effectively call arbitrary level-editing tools.

3 Methods

We propose an LLM-based agent framework for procedural content generation (PCG) that formulates level design as iterative search-based optimization over a discrete tile grid. Rather than having the LLM directly generate the level, we host the game environment as a reinforcement learning environment in which all game dynamics are encapsulated and managed. At each step, the LLM, acting as an agent, operates in a closed loop: it observes the current level state and evaluation metrics, reasons about what to improve, invokes structured tool calls to edit the level, and then the system decides whether to accept or reject the proposed changes based on a loss function. This cycle repeats until a change budget is exhausted or the agent elects to stop.

3.1 Environment

We adopt the PCG Benchmark [2] as the evaluation and problem definition backend for our experiments. Our proposed framework is tested in six different problems; parentheses give the fixed goals and the sampling space:

- **Binary** is a 2D maze of solid and empty tiles which must be fully connected; its path metric is the longest shortest path between empty tiles (fixed: fully connected regions = 1, maximize path by default; or sampled target path $\in [32, 128]$). (I.1)
- **Binary Door** is similar to the binary problem, but its path metric is measured between start and end “door” tiles along the edge of the level (fixed: fully connected regions = 1, maximize door path by default; or sampled target door path $\in [32, 128]$). (I.2)
- **Zelda**, based on [29], is a simple top-down dungeon crawler in which the player needs to be able to grab a key and reach the exit while killing enemies; its path metrics are player-to-key and key-to-door distances (fixed: 1 player/key/door, 3 enemies, maximize paths; or sampled target paths $\in [10, 40]$). (I.3)
- **Lode Runner**, based on [30], is a platformer puzzle game. The player controls a character that can move left and right and climb ladders but cannot jump. The goal is to collect all the gold without being arrested by the enemies (fixed: 1 player, 3 gold, 2 enemies, 10 ladders, 5 ropes; or sampled: ladders, ropes $\in [0, 35]$). (I.4)
- **Sokoban**, based on [31], is a top-down grid-based puzzle game. The player needs to push all the crates in the level to cover a specific target (fixed: 1 player, 3 crates/targets, solver heuristic = 0 (which means solvable); or sampled: crates $\in [2, 6]$). (I.5)
- **Super Mario Bros. (SMB)**, based on [32], is a platformer game. The player needs to traverse the level from left to right while avoiding falling into pits and avoiding enemies or killing them by jumping on top of them. Levels should have no floating enemies, and be solvable by an A* agent or a greedy agent that is always trying to jump and move right. Note that all the metrics in SMB are from real game simulations done by the player agent. (fixed: completion = 1, kills = 3, jumps = 5, coins = 3; sampled: kills $\in [0, 4]$, jumps $\in [0, 6]$, coins $\in [0, 3]$). (I.6)

These problems were selected as they span a range of complexity, from simple mazes to full platformer physics simulations. The level construction requirements are also increasing in difficulty, from simple tile counts and static pathfinding, to puzzle games with mutable states (requiring tree search), and dynamic environments with moving enemies. See Section I for full details about the setup of each problem, representation, and target metrics.

3.2 Available Tools

Our LLM agent edits levels via a structured set of tools rather than directly generating the entire map as raw tokens. Each tool call is a human-readable operation with explicit parameters, so the agent must pair its edits with a rationale and act through an auditable interface. These tools define a constrained and interpretable action space that includes both low level edit operations and higher level PCG algorithms.

The default primitive edit tool supports direct tile edits in all domains: it can place a single tile, draw a horizontal or vertical line, or fill or outline a rectangular region using the tile vocabulary of the

current game. The PCG algorithms are also exposed through the same tool interface and evaluate their effects with the same accept/reject loop. For binary maze style levels, these include independent random sampling, depth-first-search maze carving, Binary Space Partitioning room-and-corridor generation, cellular-automata smoothing, connected-component post-processing, random-walk cave digging, and Wave Function Collapse from a built-in reference. For Zelda, we additionally provide a tile-placement generator for full random layouts and reuse Binary wall-layout generators while preserving special entity tiles.

This shared interface lets the agent compose local brush strokes with coarse structural operations. Some tools overwrite much of the current layout, while others refine the current layout in place; consequently, effective optimization often requires multi-tool sequences rather than a single isolated operation. For instance, if the level is initialized as all-empty, applying cellular automata (CA) directly is typically ineffective because they rely on existing local variation. A more useful strategy is to first apply random sampling to introduce heterogeneous structure, then use CA to smooth the layout into a more coherent form. Appendix E.3 gives the detailed tool list, simplified algorithms, and default parameters.

3.3 Optimization Loss

The optimization loop follows a hill-climbing strategy in which the LLM agent iteratively proposes edits to a game level, and each candidate is evaluated by the PCG benchmark to produce a scalar score. The score is a weighted sum of 4 problem-specific components: controllable metric alignment, solvability, structural validity, change penalty. Table 1 summarizes the scoring components for each problem type.

For each controllable metric, the designer can specify either a target value or a maximization objective. Target matching penalizes deviation as $-w \cdot |x - x^*|$, while maximization rewards larger values as $+w \cdot x$, where w is the metric weight, x is the current value, and x^* is the target. Solvability contributes a large bonus or penalty depending on whether the level is playable. Structural validity terms enforce domain constraints that are prerequisites for playability but are not designer-controllable parameters, such as required entity counts; these terms apply a fixed bonus when satisfied and a penalty proportional to the deviation otherwise. Finally, a shared change penalty discourages overly disruptive edits by penalizing candidates that differ from the last accepted level by more than 60% of tiles.

By default, acceptance follows a greedy hill-climbing rule with a small exploration probability. A candidate with zero tile changes is always rejected; a candidate with higher score than the current level is accepted; and a lower-scoring candidate is accepted with probability ϵ under the greedy strategy, or with probability $\exp((s_{\text{new}} - s_{\text{cur}})/T)$ under simulated annealing, where T decays by step. Accepted steps update the current level, metrics, score, and remaining change budget, while rejected steps are retained as negative feedback in the conversation trace.

3.4 LLM Response

At each iteration, the agent receives the current level, evaluation metrics, available tools, and optional recent feedback, then returns a structured JSON response. A STEP response contains a short rationale and an ordered list of tool calls; a STOP response terminates the run. The optimizer executes the calls, evaluates the resulting candidate, and records the accept/reject outcome for subsequent turns.

When the conversation window has length $k > 1$, the prompt also includes the previous $k-1$ prompt-response pairs together with compact feedback such as score changes, metric deltas, tool-call outcomes, and accept/reject reasons. This gives the agent local trajectory memory without fine-tuning. To bound search, we track the cumulative number of tiles changed by accepted edits and stop once it reaches mHW , where $H \times W$ is the grid size and m is a configurable budget multiplier. The loop also ends if the agent stops, the maximum step count is reached, or repeated LLM errors trigger a circuit breaker.

Problem	Controllable Metrics	Solvability	Structural / Additional Terms
Binary	Longest shortest path length; number of connected regions	Connected empty tiles exist (path length > 0)	—
BinaryDoor	Path length between doors; number of connected regions	Doors are connected via doors	—
Zelda	Number of enemies; player-to-key path length; key-to-door path length; total solution length	Player can reach the key and the key can reach the door	Exactly one player, one key, and one door; exactly one connected region
Sokoban	Number of crates; number of target tiles; solution length	Automated solver finds a valid solution	Exactly one player; number of crates equals number of targets
LodeRunner	Number of gold, enemy, ladder, and rope tiles;	Exactly one player, at least one gold, and all gold reachable; penalty graduated by fraction of uncollectable gold	Exactly one player
SMB	Enemies killed; coins collected; jumps performed (all from gameplay simulation)	Simulated agent reaches the end of the level; scored continuously as completion fraction	No malformed tube formations

Table 1: Problem-specific controllable metrics, solvability requirements, and additional structural constraints used in our *optimization loss* function (not evaluation metrics like quality, diversity and controllability scores). Additionally all problem types share a graduated change penalty that activates when more than 60% of tiles differ from the last accepted level, scaling linearly with the excess change ratio.

4 Experiments

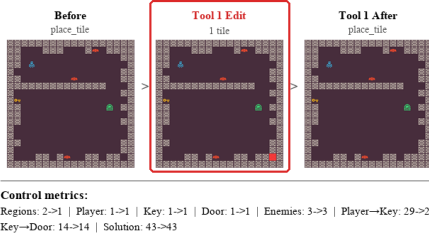
We conduct our experiments using eight LLM backbones from four model families, varying in size and release date: Claude Sonnet 4.6 [33] and Claude Opus 4.6 [34] (Anthropic); Gemini 2.5 Pro [35], Gemini 3 Flash preview [36], and Gemini 3 Pro preview [37] (Google); GPT-4o-mini [38] and o3-mini [39] (OpenAI); and Qwen 3.5-35B-A3B-FP8 [40] (Alibaba), served locally via vLLM on two NVIDIA 4090 GPUs, each with 24GB VRAM.

We evaluate the framework under three complementary forms of control. First, we run fixed-target optimization across several PCG Benchmark domains, asking agents to match predefined functional metrics such as connectivity, entity counts, path lengths, or gameplay outcomes. Second, we run sampled-target controllability experiments, where the desired metric values are resampled per trial to test whether the same agent can adapt its editing strategy to different objectives within a domain. Third, we provide free-form language instructions that specify higher-level design intent, such as challenge rooms, branching paths, or thematic Mario scenarios, while the agent still has to satisfy functional constraints. Final levels are evaluated with adapted PCG Benchmark scores for quality, controllability, and diversity; Appendix A gives the scoring definitions and trial protocol.

To isolate the role of the agentic optimization loop, we additionally run a controlled ablation package on SMB with the A* player agent using Gemini 3 Flash, optimizing to fixed targets (3 enemies kills, collect 3 coins, 5 jumps during game play). This package uses empty initialization, 30 trials, and a 100-step budget. The full agent uses the primitive edit tools, observes evaluator feedback, and retains evaluator-based accept/reject between steps. We compare it against two direct-generation baselines and two non-LLM tool-search baselines: Direct LLM one-shot asks the model to emit a complete level in a single call; Iterative Direct LLM repeatedly asks the model to rewrite the previous complete ASCII level without tool calls or evaluator feedback during generation; Random Tool Search samples primitive edit tool calls without LLM planning; and Evolutionary Tool Search evolves candidate primitive edit tool call sequences using the same optimization score as fitness. We also include ablations that remove historical feedback, metric observations in the LLM prompt, or evaluator-based accept/reject during generation. For this ablation package, we emphasize solvability, target alignment, and post-hoc structural diagnostics, because several variants obtain similarly high benchmark quality despite large differences in reliability and target-directed behavior.

Zelda - Step 7

optimization score: 503.0 -> 513.0 | 1 edited tool calls | before/edit/after



Rationale. The agent observes that the level has two connected regions instead of the target one and identifies an isolated empty tile at coordinate (15, 15) as the cause. It reasons that replacing this tile with a wall should remove the spurious region without disturbing the main playable space or the already satisfactory path lengths.

Plan. Place a wall at (15, 15), then re-evaluate the level to confirm that the region count becomes one while player, key, door, enemy, and path metrics remain unchanged.

Figure 3: **Zelda tool-edit trace.** A targeted wall placement removes an isolated region, moving the level closer to the optimization target.

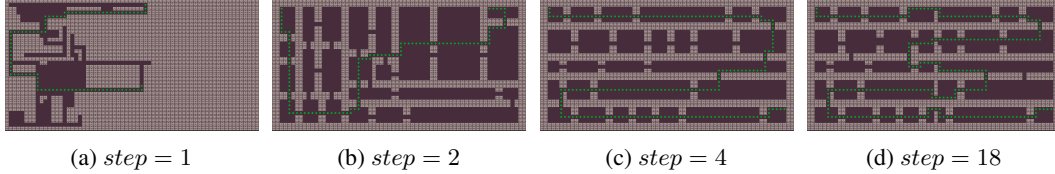


Figure 4: Iterative level editing with PCG tools. The agent combines BSP generator calls with local edits to improve path length.

5 Results

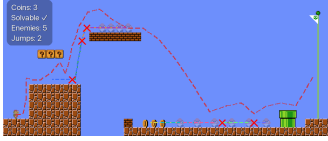
Across the evaluated domains, the generated levels suggest that LLM agents can produce plausible, functional content from symbolic level descriptions alone (Figs. 1 and 2). In Binary and Binary Door, agents reliably discover long-path maze patterns, adapting the familiar zig-zag strategy to door constraints. In Lode Runner, Zelda, and Sokoban, the same loop handles object placement and reachability constraints in more semantically loaded layouts, although Lode Runner scores remain sensitive to the benchmark’s human-trajectory prior. SMB is harder: models often produce solvable but conservative low, flat levels, yet can still manipulate pits, ceilings, and enemy placement to induce target jumps and kills. These examples suggest broad qualitative competence, while also showing that richer game mechanics and stricter functional constraints make target matching substantially less forgiving.

During this process, the tool-use traces show more than blind metric search. Although the models receive no multimodal input and operate on flattened one-dimensional textual representations of the level, their rationales and plans often reveal spatial reasoning over the underlying grid. Figure 3 gives a compact example: the Zelda agent identifies a single isolated empty tile as the source of an extra connected region, then proposes a one-tile wall edit that preserves the playable dungeon. Additional tool-edit traces examples are provided in Appendix C.

The same tool-mediated formulation also extends naturally beyond primitive tile edits. When a domain-specific PCG algorithm is available, it can be exposed as another callable action inside the agent loop. Figure 4 shows this in Binary Maze: the agent calls a Binary Space Partitioning generator to make large structural changes, then uses smaller edits to tune the final path. This suggests that the framework is extensible: PCG algorithms can contribute useful inductive biases, while the evaluator still decides whether a proposed algorithmic edit actually helps the current level.

We also test natural language as an additional instruction channel for the LLM agent (Fig. 5). These prompts produce interpretable effects, such as Zelda challenge rooms and branching paths, or Mario levels organized around Goomba-themed scenarios. The results are not perfect: the agent often simplifies the requested design, and functional objectives can still dominate the final layout. Nevertheless, Fig. 5 shows that the same agentic loop can accept qualitative design intent alongside metric feedback, which is precisely the kind of mixed control that standard functional benchmarks only partially measure.

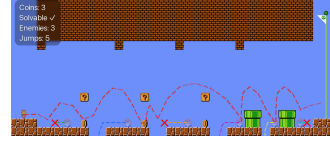
Table 2 separates three sources of control in Agentic PCG. Within the agentic variants, removing historical feedback or metric observations has only a modest effect as long as evaluator-gated iteration



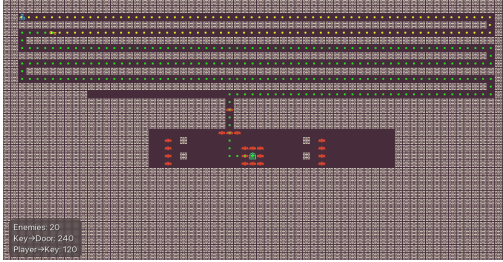
(a) **Goomba Cliff Dive.** “Make a level in which Mario jumps off a cliff and bounces off a series of falling Goombas.” The agent partially realizes the bounce-chain setup.



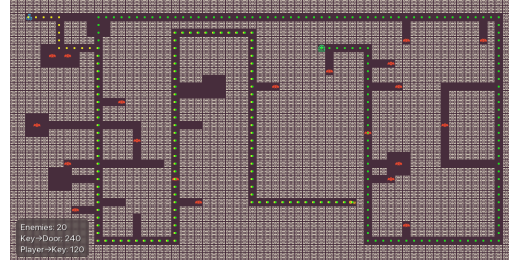
(b) **Goomba Household.** “Make a level that includes a cross-section of a two-storey Goomba home. Have two Goombas around a table on the main floor.” Two Goombas are placed around a simple table-like structure.



(c) **Goomba Cave.** “Underground cave filled with Goombas.” The prompt yields a roofed, Goomba-heavy Mario layout.



(d) **Challenge Room.** “Generate a level with a challenge room.” The agent creates an enemy-dense room along the path to the door.



(e) **Branching Path.** “Generate a level with a branching path.” The agent creates a maze-like dungeon with multiple route choices.

Figure 5: Freeform instructions produce interpretable design changes while the agent still prioritizes functional constraints.

Table 2: Compact SMB A* ablation and baseline results with Gemini 3 Flash. All rows use empty initialization, 30 trials, and a 100-step budget unless noted. Target alignment summarizes fixed-target matching, while coherence and pattern divergence are recomputed post hoc from final levels; lower pattern KL indicates closer local tile-pattern statistics to the SMB human-designed reference corpus. Full results are in Appendix F.1.

Method	Solvable	Coherence $\uparrow$	Target Align. $\uparrow$	Diversity $\uparrow$	Pat. KL $\downarrow$
Full agent (Our Method)	30/30	0.83	0.74	0.93	0.87
No historical feedback	30/30	0.83	0.72	0.90	0.94
No metric observation	29/30	0.91	0.76	0.93	0.90
No evaluation loop	21/30	0.92	0.63	0.86	0.93
Iterative direct LLM, 20 steps	30/30	0.87	0.46	1.00	0.71
Direct LLM one-shot	25/30	0.88	0.35	0.92	0.69
Random tool search	9/30	0.70	0.20	1.00	1.58
Evolutionary tool search	29/30	0.68	0.94	1.00	3.77

remains in place. The larger drop appears when the evaluation loop itself is removed, suggesting that reliability comes less from prompt context alone than from the harness of the framework, including metric evaluation, rollback, and iterative search.

The baselines expose the complementary failure modes. Direct LLM generation preserves local Mario-like structure but weakens control over simulated gameplay outcomes, while pure tool-space search can satisfy explicit targets more aggressively at the cost of coherence and reference-like organization. The full agent sits between these extremes: it uses evaluator feedback to ground functional optimization while retaining the LLM’s design priors and language-steerable control.

Game design is ultimately subjective, and high-level intentions such as style, pacing, theme, or intended player experience are difficult to fully capture with fixed quantitative benchmark scores. We nevertheless provide a quantitative analysis using our adapted PCG Benchmark evaluation protocol; full cross-model scores and additional model-level analysis are reported in Appendix A.

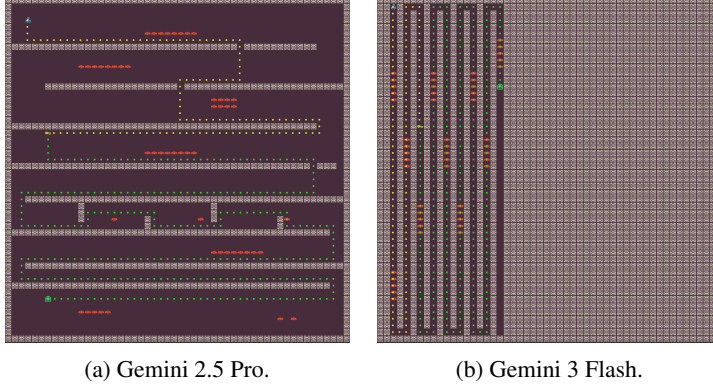


Figure 6: Example comparison of two LLM agents on the same 50×50 Zelda optimization problem with identical all-empty initialization and target metrics. Although both runs satisfy nearly the same functional objectives, they exhibit markedly different level editing behaviors and optimization trajectories.

6 Discussion

Table 2 clarifies what is doing the work in Agentic PCG. Within the agentic variants, historical feedback and metric observations change performance only modestly as long as evaluator-gated iteration remains in place, while removing metric evaluation and rollback causes a much larger drop in solvability and target alignment. This suggests that reliability comes from the full optimization harness rather than from prompt context alone. The baseline rows expose the complementary failure modes: direct LLM generation preserves local Mario-like structure but misses simulated gameplay targets, whereas non-LLM tool search can optimize explicit metrics more aggressively while producing less coherent and less reference-like layouts.

The value of Agentic PCG is therefore not unconditional dominance over every search baseline. Rather, it occupies a middle ground between purely functional search and unguided LLM generation. The framework couples language-conditioned planning with executable edit tools, metric feedback, and evaluator-based accept/reject, allowing the model to optimize measurable constraints while retaining some of the structural priors that make levels appear organized. Figure 6 reinforces this point: even when aggregate scores are similar, different LLM backbones can follow substantially different edit trajectories and produce visibly different layouts, so final benchmark metrics alone are an incomplete description of the generation process.

A remaining limitation is that accepted edits are still determined by a fixed optimization loss. That loss covers the user-specified target metrics and solvability checks, but not the full space of design goals a human designer may care about. Replacing the loss with an unconstrained LLM judge would risk over-weighting free-form language preference and weakening functional guarantees; relying only on metric matching can make the search too rigid. A useful next step is therefore to integrate richer, explicit design evaluators or preference models into the same tool-mediated loop, so that functional constraints and open-ended design intent can be traded off in a more controlled way.

7 Conclusion

We introduced Agentic PCG, a tool-mediated framework for using LLMs to generate and refine grid-based game levels. Compared with approaches that train or fine-tune models to emit levels tile by tile [16, 15], our workflow lets recent reasoning models adapt across domains by acting through existing PCG infrastructure [2]: classical generators, search procedures, evaluators, and low-level editing operations such as tile placement, wall drawing, and flood fill.

Our results suggest that this agentic formulation can satisfy quantitative targets while preserving more structured design behavior. The agents use evaluation feedback not only to match numeric proxies such as difficulty, but also to scaffold higher-order gameplay constraints, producing features such as challenge rooms (Fig. 5d), branching paths (Fig. 5e), forced traversal, and backtracking. In this view, traditional PCG methods are not displaced by LLMs; they become reusable, inspectable tools that the agent can compose within a broader editing and optimization loop.

Looking forward, the promise of Agentic PCG lies in making this tool ecosystem richer and more inclusive. Future systems could expose agents to a wider range of generators, simulators, learned

evaluators, playtesters, constraint solvers, and human-facing editor tools, while also allowing agents to synthesize new evaluation or design tools from free-form user objectives [41]. Such systems would move beyond prompting models to generate content directly, toward LLM agents that coordinate diverse design knowledge and computational tools to support more expressive, controllable, and open-ended procedural content generation.

References

- [1] N. Shaker, J. Togelius, and M. J. Nelson, “Procedural content generation in games,” 2016.
- [2] A. Khalifa, R. Gallotta, M. Barthet, A. Liapis, J. Togelius, and G. N. Yannakakis, “The procedural content generation benchmark: An open-source testbed for generative challenges in games,” in *Proceedings of the 20th International Conference on the Foundations of Digital Games*, ser. FDG ’25. New York, NY, USA: Association for Computing Machinery, 2025. [Online]. Available: <https://doi.org/10.1145/3723498.3723794>
- [3] J. Togelius, G. N. Yannakakis, K. O. Stanley, and C. Browne, “Search-based procedural content generation: A taxonomy and survey,” *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 3, no. 3, pp. 172–186, 2011.
- [4] R. Gallotta, K. Arulkumaran, and L. B. Soros, “Evolving spaceships with a hybrid l-system constrained optimisation evolutionary algorithm,” in *Proceedings of the Genetic and Evolutionary Computation Conference Companion*, 2022, pp. 711–714.
- [5] A. Khalifa, M. C. Green, G. Barros, and J. Togelius, “Intentional computational level design,” in *Proceedings of The Genetic and Evolutionary Computation Conference*, 2019, pp. 796–803.
- [6] N. Shaker, M. Nicolau, G. N. Yannakakis, J. Togelius, and M. O’neill, “Evolving levels for super mario bros using grammatical evolution,” in *2012 IEEE Conference on Computational Intelligence and Games (CIG)*. IEEE, 2012, pp. 304–311.
- [7] M. Cook, S. Colton, A. Raad, and J. Gow, “Mechanic miner: Reflection-driven game mechanic discovery and level design,” in *European Conference on the Applications of Evolutionary Computation*. Springer, 2013, pp. 284–293.
- [8] C. B. Browne, “Automatic generation and evaluation of recombination games,” Ph.D. dissertation, Queensland University of Technology, 2008.
- [9] A. Summerville, S. Snodgrass, M. Guzdial, C. Holmgård, A. K. Hoover, A. Isaksen, A. Nealen, and J. Togelius, “Procedural content generation via machine learning (pcgml),” *IEEE Transactions on Games*, vol. 10, no. 3, pp. 257–270, 2018.
- [10] S. Dahlskog, J. Togelius, and M. J. Nelson, “Linear levels through n-grams,” in *Proceedings of the 18th International Academic MindTrek Conference: Media Business, Management, Content & Services*, 2014, pp. 200–206.
- [11] S. Snodgrass and S. Ontanón, “Learning to generate video game maps using markov models,” *IEEE transactions on computational intelligence and AI in games*, vol. 9, no. 4, pp. 410–422, 2016.
- [12] A. Summerville and M. Mateas, “Super mario as a string: Platformer level generation via lstms,” *arXiv preprint arXiv:1603.00930*, 2016.
- [13] A. Sarkar, Z. Yang, and S. Cooper, “Conditional level generation and game blending,” *arXiv preprint arXiv:2010.07735*, 2020.
- [14] V. Volz, J. Schrum, J. Liu, S. M. Lucas, A. Smith, and S. Risi, “Evolving mario levels in the latent space of a deep convolutional generative adversarial network,” in *Proceedings of the genetic and evolutionary computation conference*, 2018, pp. 221–228.
- [15] G. Todd, S. Earle, M. U. Nasir, M. C. Green, and J. Togelius, “Level generation through large language models,” in *Proceedings of the 18th International Conference on the Foundations of Digital Games*, 2023, pp. 1–8.

- [16] S. Sudhakaran, M. González-Duque, M. Freiburger, C. Glanois, E. Najarro, and S. Risi, “Mariogpt: Open-ended text2level generation through large language models,” *Advances in Neural Information Processing Systems*, vol. 36, pp. 54 213–54 227, 2023.
- [17] T. Merino, S. Earle, R. Sudhakaran, S. Sudhakaran, and J. Togelius, “Making new connections: Llms as puzzle generators for the new york times’ connections word game,” in *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, vol. 20, no. 1, 2024, pp. 87–96.
- [18] S. Huang, “Word2minecraft: Generating 3d game levels through large language models,” Master’s thesis, New York University Tandon School of Engineering, 2025.
- [19] M. U. Nasir, Y. Li, S. James, and J. Togelius, “Mortar: Evolving mechanics for automatic game design,” *arXiv preprint arXiv:2601.00105*, 2025.
- [20] S. Earle, A. Khalifa, M. U. Nasir, Z. Jiang, G. Todd, A. Banburski-Fahey, and J. Togelius, “Scriptdoctor: Automatic generation of puzzlescript games via large language models and tree search,” in *2025 IEEE Conference on Games (CoG)*. IEEE, 2025, pp. 1–5.
- [21] R. Gallotta, A. Liapis, and G. Yannakakis, “Llmaker: A game level design interface using (only) natural language,” in *2024 IEEE Conference on Games (CoG)*. IEEE, 2024, pp. 1–2.
- [22] R. Gallotta, A. Liapis, and G. N. Yannakakis, “Freyr: A framework for recognizing and executing your requests,” *arXiv preprint arXiv:2501.12423*, 2025.
- [23] A. Liapis, G. Smith, and N. Shaker, “Mixed-initiative content creation,” in *Procedural content generation in games*. Springer, 2016, pp. 195–214.
- [24] A. Khalifa, P. Bontrager, S. Earle, and J. Togelius, “Pcgrl: Procedural content generation via reinforcement learning,” in *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, vol. 16, no. 1, 2020, pp. 95–101.
- [25] S. Earle, M. Edwards, A. Khalifa, P. Bontrager, and J. Togelius, “Learning controllable content generators,” in *2021 IEEE Conference on Games (CoG)*, 2021, pp. 1–9.
- [26] Z. Jiang, S. Earle, M. Green, and J. Togelius, “Learning controllable 3d level generators,” in *Proceedings of the 17th International Conference on the Foundations of Digital Games*, ser. FDG ’22. New York, NY, USA: Association for Computing Machinery, 2022. [Online]. Available: <https://doi.org/10.1145/3555858.3563273>
- [27] S. Earle, Z. Jiang, and J. Togelius, “Scaling, control and generalization in reinforcement learning level generators,” in *2024 IEEE Conference on Games (CoG)*, 2024, pp. 1–8.
- [28] S. Earle, Z. Jiang, E. Vinitzky, and J. Togelius, “Video game level design as a multi-agent reinforcement learning problem,” *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, vol. 21, no. 1, pp. 32–42, Nov. 2025. [Online]. Available: <https://ojs.aaai.org/index.php/AIIDE/article/view/36807>
- [29] T. T. Shigeru Miyamoto, “The legend of zelda,” Nintendo Entertainment System, 1986, video game.
- [30] D. Smith, “Lode runner,” Apple II, 1983, pC.
- [31] T. Rabbit, “Sokoban,” PC, 1982.
- [32] T. T. Shigeru Miyamoto, “Super mario bros.” Nintendo Entertainment System, 1985, video game.
- [33] Anthropic, “Claude sonnet 4.6 system card,” <https://www-cdn.anthropic.com/bbd8ef16d70b7a1665f14f306ee88b53f686aa75.pdf>, Feb. 2026, accessed: 2026-03-24.
- [34] —, “Claude opus 4.6 system card,” <https://www-cdn.anthropic.com/6a5fa276ac68b9aeb0c8b6af5fa36326e0e166dd.pdf>, Feb. 2026, accessed: 2026-03-24.

- [35] G. Comanici, E. Bieber, M. Schaekermann, I. Pasupat, N. Sachdeva, I. Dhillon, M. Blistein, O. Ram, D. Zhang, E. Rosen *et al.*, “Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities,” *arXiv preprint arXiv:2507.06261*, 2025.
- [36] Google DeepMind, “Gemini 3 flash model card,” <https://storage.googleapis.com/deepmind-media/Model-Cards/Gemini-3-Flash-Model-Card.pdf>, Dec. 2025, published: December 2025. Accessed: 2026-03-24.
- [37] —, “Gemini 3 pro model card,” <https://storage.googleapis.com/deepmind-media/Model-Cards/Gemini-3-Pro-Model-Card.pdf>, Dec. 2025, model card update: December 2025. Accessed: 2026-03-24.
- [38] OpenAI, “Gpt-4o system card,” <https://cdn.openai.com/gpt-4o-system-card.pdf>, Aug. 2024, accessed: 2026-03-24.
- [39] —, “Openai o3-mini system card,” <https://cdn.openai.com/o3-mini-system-card-feb10.pdf>, Jan. 2025, accessed: 2026-03-24.
- [40] Qwen Team, “Qwen3.5: Towards native multimodal agents,” February 2026. [Online]. Available: <https://qwen.ai/blog?id=qwen3.5>
- [41] K. Ellis, C. Wong, M. Nye, M. Sablé-Meyer, L. Morales, L. Hewitt, L. Cary, A. Solar-Lezama, and J. B. Tenenbaum, “Dreamcoder: Bootstrapping inductive program synthesis with wake-sleep library learning,” in *Proceedings of the 42nd acm sigplan international conference on programming language design and implementation*, 2021, pp. 835–850.
- [42] S. M. Lucas and V. Volz, “Tile pattern kl-divergence for analysing and evolving game levels,” in *Proceedings of the Genetic and Evolutionary Computation Conference*, ser. GECCO ’19. New York, NY, USA: Association for Computing Machinery, 2019, p. 170–178. [Online]. Available: <https://doi.org/10.1145/3321707.3321781>

Appendix Contents

- PCG Benchmark Evaluation
- Additional Discussions
 - World-Model-Like Reasoning in the Tool Loop
- Tool-Edit Traces and Spatial Reasoning
- Gemini 2.5 Pro Tool-Use Case Studies
- Tool Implementation Details
 - Tool Interface and Dispatch
 - Primitive Edit Tools
 - PCG Algorithms as Tools
 - Evaluation, Rollback, and Acceptance
- Additional Experiment Results
 - Full SMB Ablation Table
- Structural Coherence Metric
- Pattern Divergence Metrics
- Environment Details
 - Binary Maze
 - Binary Maze with Doors
 - Zelda
 - Lode Runner
 - Sokoban
 - Super Mario Bros
- More Qualitative Results
 - Binary
 - Binary Door
 - Lode Runner
 - Super Mario Bros (A*)
 - Super Mario Bros (A*) – Evolutionary Tool Search in the Ablation
 - Super Mario Bros (Auto)
- Prompt Details

A PCG Benchmark Evaluation

We evaluate final levels using an adapted version of the PCG Benchmark [2] along three axes: *quality*, *controllability*, and *diversity*. For each game-specific metric value x , we use a trapezoidal normalization

$$T(x; l, a, b, u) = \begin{cases} 0, & x \leq l \text{ or } x \geq u, \\ \frac{x-l}{a-l}, & l < x < a, \\ 1, & a \leq x \leq b, \\ \frac{u-x}{u-b}, & b < x < u, \end{cases}$$

where $[a, b]$ is the target interval and $[l, u]$ is the valid scoring range. Quality averages T over fixed functional targets, while controllability uses the same form with targets sampled from the control space and tolerance intervals defining $[a, b]$. Diversity is computed pairwise over generated levels using task-specific distances, including structural differences such as Hamming distance and behavioral differences such as solution paths, exploration maps, or visited tiles. These evaluation scores are computed only on the final optimized level and are distinct from the optimization score used for accepting or rejecting actions during generation.

Although our framework supports all six environments, we run the main sweep on four representative domains: Binary, Binary Door, Lode Runner, and Super Mario Bros with both A* and greedy agents

Table 3: Cross-model comparison across five game domains. Solv. reports solvable quality/diversity trials; Exact reports controllability trials whose final metrics exactly match the sampled target. Best metric scores per game are in **bold**.

Game	Model	Qual $\uparrow$	Solv.	Divers $\uparrow$	Control $\uparrow$	Exact
Binary	Claude Sonnet 4.6	0.9970	20/20	0.1500	0.9459	5/20
	Claude Opus 4.6	1.0000	20/20	0.1000	0.8319	2/20
	Gemini 2.5 Pro	1.0000	19/19	0.3684	0.9939	17/20
	Gemini 3 Flash	1.0000	20/20	0.1500	1.0000	20/20
	Gemini 3 Pro	1.0000	17/17	0.1176	1.0000	20/20
	GPT-4o Mini	0.8329	20/20	0.4500	0.6566	3/20
	o3-mini	0.9939	20/20	0.3000	0.9008	9/20
	Qwen 3.5 35B	0.9903	40/40	0.3500	0.7718	2/20
Binary Door	Claude Sonnet 4.6	0.9805	8/20	0.5000	0.6943	0/20
	Claude Opus 4.6	0.9687	10/20	0.4000	0.7189	0/20
	Gemini 2.5 Pro	1.0000	9/14	0.3333	0.7372	3/18
	Gemini 3 Flash	1.0000	12/20	0.2500	0.8079	3/20
	Gemini 3 Pro	1.0000	13/15	0.1538	0.8186	5/20
	GPT-4o Mini	0.8867	8/20	0.2500	0.4931	0/20
	o3-mini	0.9330	7/20	0.5714	0.5825	1/20
	Qwen 3.5 35B	0.8516	18/40	0.2778	0.4251	0/20
Lode Runner	Claude Sonnet 4.6	0.2344	17/20	0.0588	0.9898	0/20
	Claude Opus 4.6	0.2292	20/20	0.0500	0.9975	6/20
	Gemini 2.5 Pro	0.2292	20/20	0.0500	0.9979	19/20
	Gemini 3 Flash	0.2615	20/20	0.0500	1.0000	20/20
	Gemini 3 Pro	0.2615	20/20	0.0500	1.0000	20/20
	GPT-4o Mini	0.2292	8/20	0.1250	0.9992	4/20
	o3-mini	0.2292	17/20	0.0588	0.9982	9/20
	Qwen 3.5 35B	0.2284	29/40	0.0345	0.9987	3/20
SMB (A*)	Claude Sonnet 4.6	0.9975	20/20	0.8500	0.8061	7/20
	Claude Opus 4.6	0.9968	20/20	0.8500	0.7805	4/20
	Gemini 2.5 Pro	0.9978	20/20	0.8500	0.8115	9/20
	Gemini 3 Flash	0.9973	20/20	0.9500	0.7568	13/20
	Gemini 3 Pro	0.9977	20/20	0.9500	0.8262	16/20
	GPT-4o Mini	0.9988	10/20	1.0000	0.7665	2/20
	o3-mini	0.9955	14/20	0.9286	0.8148	7/20
	Qwen 3.5 35B	0.9948	32/40	0.9375	0.7933	5/20
SMB (Greedy)	Claude Sonnet 4.6	0.9978	20/20	0.9000	0.9713	3/20
	Claude Opus 4.6	0.9975	20/20	0.8500	0.9464	2/20
	Gemini 2.5 Pro	0.9979	20/20	0.8500	0.9701	7/20
	Gemini 3 Flash	0.9971	20/20	0.9000	0.9878	8/20
	Gemini 3 Pro	0.9976	20/20	0.9500	0.9760	9/20
	GPT-4o Mini	0.9990	10/20	1.0000	0.9511	2/20
	o3-mini	0.9951	16/20	0.9375	0.9437	5/20
	Qwen 3.5 35B	0.9967	33/40	0.8182	0.9176	3/20

as player agents, respectively. For quality, we use 20 controlled-seed trials with fixed challenging targets, averaging quality across trials and computing diversity across all solvable final levels. For controllability, we use 10 controlled-seed trials with randomly sampled targets shared across models,² averaging over solvable final levels. Trials that fail because of repeated server-side API errors or timeouts are discarded.

Table 3 expands the aggregate benchmark scores with two stricter diagnostics. The solvability column shows that high metric scores are not simply artifacts of averaging over many failed levels: most frontier models solve nearly all Binary, Lode Runner, and SMB trials, while Binary Door remains less reliable because door placement adds a hard structural constraint. The exact-match column is more selective than the trapezoidal controllability score, and highlights that models can receive high control scores while still missing the sampled target by a small amount, especially in SMB where gameplay dynamics make small tile changes affect jumps, coins, and enemy kills nonlinearly.

The sweep also shows that smaller local models can still function effectively within our framework on benchmark-based objectives: Qwen3.5-35B-A3B follows the loss-based optimization harness, adheres to the required JSON output format, and optimizes toward the selected target metrics. However, larger frontier models appear more consistent in qualitative examples along less explicitly specified dimensions, such as playability, interestingness, and aesthetic organization, which are not fully captured by the benchmark metrics.

²All randomness is seeded, so the sampled control targets are the same across models.

B Additional Discussions

B.1 World-Model-Like Reasoning in the Tool Loop

One possible explanation for the qualitative difference between Agentic PCG and purely functional search is that good level editing requires some ability to anticipate game dynamics, such as enemy movement, player traversal, collisions, and local affordances. In this sense, the LLM does not merely map a level state to an edit action. It repeatedly proposes edits, observes evaluator feedback from PCG Benchmark, and adjusts its next plan in response to the measured consequences of those edits.

We do not claim that the LLM learns or contains an explicit simulator of the game engine. Rather, the tool-mediated loop gives the model a grounded setting in which prior knowledge about game structure, natural-language planning, and executable feedback can interact. This may help explain why the agent can sometimes preserve recognizable design patterns while still optimizing toward functional targets, whereas smaller trained policies or non-LLM search procedures may satisfy the same metrics through less interpretable editing behavior.

C Tool-Edit Traces and Spatial Reasoning

Figures 7–10 show additional representative accepted optimization steps with the individual tool edits highlighted in red. These examples further illustrate a useful property of the agentic formulation: although the policy model is a text-only LLM and observes the level through symbolic grids, metric feedback, and tool descriptions rather than pixels, it can still use prior knowledge about game structure to reason spatially. Across domains, the model identifies isolated regions, blocked passages, bottlenecks, traversal affordances, and object-placement constraints, then converts that reasoning into localized tool calls. This does not mean the LLM has perfect geometric perception; rather, the tool-mediated loop gives it a grounded interface in which language-level planning, spatial priors, and executable edits can reinforce one another.

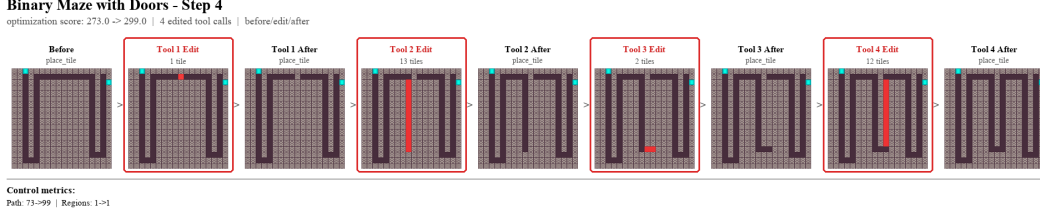


Figure 7: **Binary maze with doors tool-edit trace. Rationale:** The agent notices that the door-to-door path is already long but that a large unused central wall block can support an additional detour. It proposes to break the direct upper corridor and force the path downward, across, and back upward, increasing the path length while preserving a single connected region. **Plan:** Block the direct top-row path at (0, 8), carve a vertical path down column 7, carve a horizontal connector at row 13, and carve a vertical return path up column 9 to reconnect to the top corridor.

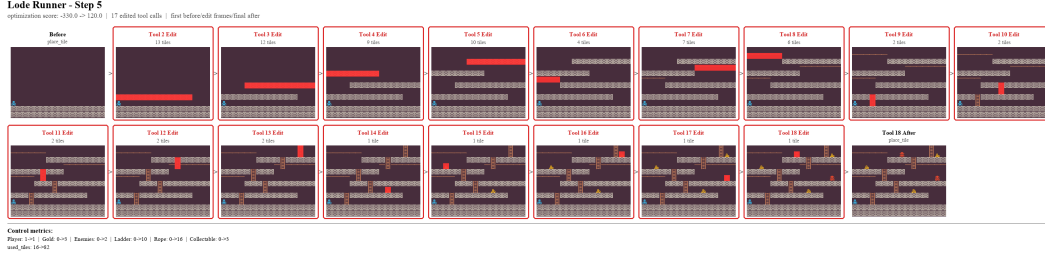


Figure 8: **Lode Runner tool-edit trace. Rationale:** Starting from an empty, unplayable level, the agent reasons that it must first create a complete traversal scaffold: platforms, ladders, ropes, gold, and enemies. This step builds a simple but functional layout that satisfies the main object and traversal requirements. **Plan:** Clear the area above the ground floor, create staggered solid platforms, add ropes for horizontal traversal, add ladders between platform levels, place three reachable gold pieces, and place two enemies on platforms.

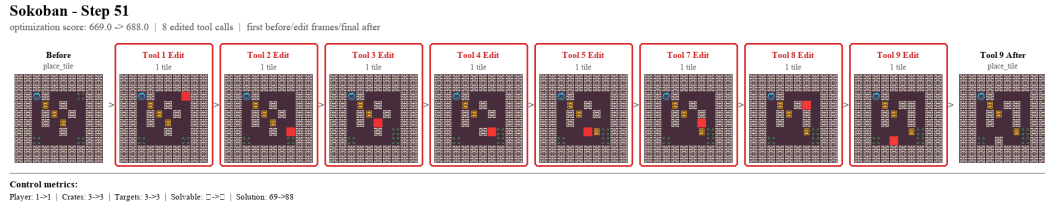


Figure 9: **Sokoban tool-edit trace. Rationale:** To increase solution length, the agent plans a more restrictive layout that forces crate movement in a more specific order. It relocates one target and one crate, adds bottleneck walls, and opens selected walls so that the puzzle remains solvable while requiring longer navigation. **Plan:** Move the target from (1, 6) to (5, 6), move the crate from (4, 4) to (5, 5), open cells at (5, 4) and (4, 3), add walls at (4, 5), (2, 5), and (6, 3), and preserve access so the player can still push all three crates onto all three targets.

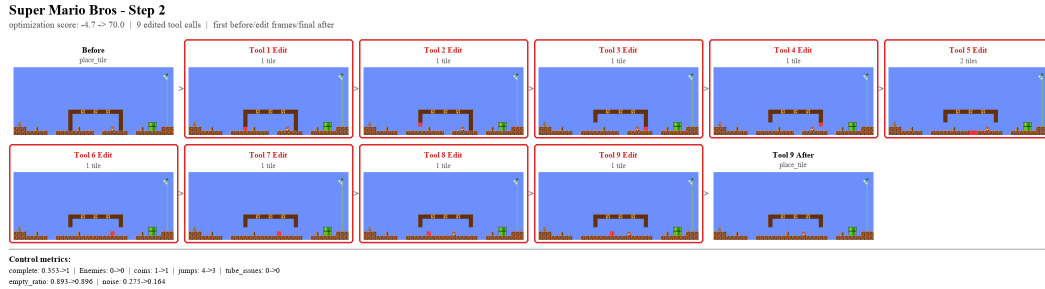


Figure 10: **Super Mario Bros tool-edit trace under an additional natural-language instruction.** The additional instruction asks for a Hansel-and-Gretel-themed Mario level in which Mario enters a gingerbread house, avoids the witch, and exits through the oven. **Rationale:** The agent recognizes that the brick house walls block Mario’s path, so it opens a two-tile-high entrance and exit, fills the middle gap to support traversal and the Goomba, and repositions collectibles and the enemy onto the main route. **Plan:** Clear entrance bricks at (13, 10) and (14, 10), clear exit bricks at (13, 22) and (14, 22), fill the floor gap at columns 16–17, move the Goomba toward the center of the house, move the coin inside the house, and preserve the remaining gaps and tube to match the target jump structure.

D Gemini 2.5 Pro Tool-Use Case Studies

The 50×50 Zelda example in Figure 6 provides a complementary view of Gemini 2.5 Pro’s tool-use behavior. In this task, Gemini 2.5 Pro and Gemini 3 Flash reach nearly the same final outcome in

terms of functional target matching, but they do so through very different edit trajectories and with substantially different final layouts. Gemini 3 Flash reaches the maximum score of 750 in only 5 steps, with 3 accepted edits (60% acceptance rate), whereas Gemini 2.5 Pro requires 100 steps to reach 749, with 8 accepted edits (8% acceptance rate). The main difference lies in edit scale: Gemini 3 Flash performs a large initial rewrite, changing 2,075 tiles (83% of the map) and improving the score from −1172 to 377 in one step, then reaches the optimum with a small 20-tile refinement. In contrast, Gemini 2.5 Pro improves the level more gradually, with its largest accepted edits changing only about 323–376 tiles.

Tool usage shows a similar pattern. Gemini 2.5 Pro makes 269 total tool calls (178 line, 88 single, 2 rect.), while Gemini 3 Flash uses only 30 (21 line, 7 single, 1 rect.). Although both agents primarily rely on line-based tile placement, Gemini 2.5 Pro uses substantially more tool calls and improves the level through smaller local modifications, whereas Gemini 3 Flash uses far fewer calls and benefits from a single large rectangular edit that rapidly imposes global structure.

To better understand how higher-level PCG tools affect the agentic editing loop, we analyze a Binary Maze sweep with Gemini 2.5 Pro. The sweep compares a primitive-edit setting, where the agent can only use `place_tile`, against a tool-augmented setting, where the agent can use `place_tile`, `generate_bsp`, `generate_ca`, and `generate_connect`. The sweep includes quality, controllability, and diversity objectives; empty and weighted initialization; greedy, epsilon-greedy, and simulated-annealing acceptance; conversation windows of length 1 and 4; and 10 trials per condition. All 72 experiment combinations completed successfully.

Table 4: Gemini 2.5 Pro Binary Maze tool-use case study. The primitive-edit condition exposes only `place_tile`; the tool-augmented condition additionally exposes Binary Space Partitioning, cellular automata, and connectivity repair tools.

Task	Primitive edit	Tool-augmented	Delta	Main readout
Quality	0.997	1.000	+0.003	Both settings are saturated; tools preserve solution quality.
Controllability	0.996	0.954	-0.043	Primitive edits are better for precise target matching.
Diversity	0.377	0.500	+0.123	Higher-level tools improve structural variety.

Table 4 shows the central tradeoff. Adding higher-level PCG tools preserves near-perfect quality and improves diversity, but reduces controllability. This suggests that structured tools help the model make larger semantic edits, while primitive tile edits remain better suited for fine-grained target matching.

Table 5: Edit acceptance and tool behavior in the Gemini 2.5 Pro Binary Maze sweep. Acceptance is computed per edit-tool call. When multiple edit calls appear in the same optimization step, each call inherits the step-level accepted/rejected outcome. Observation-only calls such as `calculate_stats` are excluded.

Context	Tool	Calls	Tool success	Acceptance rate	Avg. tiles changed / call	Avg. score delta / call
Primitive-only	<code>place_tile</code>	32357	99.9%	26.8%	5.55	-30.55
Tool-augmented	<code>place_tile</code>	33964	100.0%	20.2%	3.48	-25.69
Tool-augmented	<code>generate_bsp</code>	2299	31.0%	13.5%	32.18	3.28
Tool-augmented	<code>generate_ca</code>	298	100.0%	5.4%	22.22	-17.79
Tool-augmented	<code>generate_connect</code>	737	100.0%	45.3%	11.04	26.80

Table 5 shows that the higher-level tools are not uniformly useful. The strongest individual tool is `generate_connect`: it has the highest acceptance rate and the largest positive average score delta. This matches the structure of Binary Maze, where repairing disconnected empty regions is often the dominant failure mode. By contrast, `generate_ca` succeeds at execution time but is rarely accepted, indicating that its smoothing operations are poorly aligned with the optimization objective in this domain. `generate_bsp` produces much larger edits and can act as a structural reset, but it has a low execution success rate in this small-map setting.

The primitive-only `place_tile` condition has a higher acceptance rate than `place_tile` inside the tool-augmented condition. One interpretation is that, when higher-level tools are available, tile edits are used more often for local probing and cleanup, while the main structural gains come from specialized tools such as `generate_connect`.

Table 6: Reasons for rejected edit-tool calls in the Gemini 2.5 Pro Binary Maze sweep. Percentages are computed within rejected calls for each tool.

Tool	Rejected calls	Rejection rate	Score decreased	Tie/no improvement	No change	Tool failed	Other
<code>place_tile</code>	50810	76.6%	79.1%	10.5%	10.4%	0.0%	0.0%
<code>generate_bsp</code>	1989	86.5%	19.6%	0.6%	0.0%	79.8%	0.0%
<code>generate_ca</code>	282	94.6%	81.6%	2.5%	16.0%	0.0%	0.0%
<code>generate_connect</code>	403	54.7%	58.1%	5.0%	37.0%	0.0%	0.0%

The rejection reasons in Table 6 further separate tool execution failures from optimization failures. Most rejected `place_tile` calls are genuine harmful local edits: 79.1% of rejected calls decrease the score. This is consistent with local search around already strong solutions, where many small perturbations break connectivity or shorten the path. `generate_ca` has a similar pattern, with 81.6% of rejected calls decreasing the score.

`generate_bsp` is different: 79.8% of its rejected calls are tool execution failures. Thus, much of its rejection rate reflects implementation or parameter brittleness rather than the optimizer rejecting valid BSP-generated candidates. `generate_connect` has the lowest rejection rate among the non-primitive tools, and many of its rejected calls are no-ops. This is expected because the tool is only useful when the current level has multiple connected regions; once the map is already connected, it often has nothing to repair.

Overall, this case study supports a more nuanced interpretation of tool use. Higher-level PCG tools are valuable when their inductive bias matches a common structural bottleneck, as with connectivity repair in Binary Maze. However, coarse or weakly aligned tools can either fail frequently, as with BSP in this setting, or produce valid but low-scoring edits, as with cellular automata. Tool use therefore improves the agent not simply by expanding the action space, but by providing operations whose granularity and structural assumptions match the optimization landscape.

E Tool Implementation Details

This appendix section describes how the editing and PCG tools used by the agent are implemented. The main paper describes the tool interface at a conceptual level; here we give the concrete implementation details needed to reproduce the action space and the tool-mediated optimization loop.

E.1 Tool Interface and Dispatch

Each callable operation is implemented as a `Tool` object with three public components: a unique name, a natural-language description, and a JSON-schema-style parameter specification. Before each agent turn, the optimizer serializes the registered tool set into the prompt by listing the tool name, description, required parameters, optional parameters, and parameter types. The LLM must then return a JSON object of type `STEP`, containing a rationale and an ordered list of tool calls. Each call specifies `tool_name` and a dictionary of parameters. This design keeps the action space explicit and makes every proposed edit auditable.

Tools are stored in a `ToolRegistry`. During execution, the registry dispatches each call by name and returns a structured `ToolResult` containing a success flag, a data dictionary, and any errors. Unknown tools and exceptions are converted into failed tool results rather than crashing the optimization loop. The optimizer executes tool calls sequentially in the order emitted by the LLM, up to the configured maximum number of calls per step (20 in the main experiments). If the LLM emits more calls than the budget, the remaining calls are dropped and the trace records a truncation error.

E.2 Primitive Edit Tools

The principal low-level tool is `place_tile`. It supports three modes. In `single` mode, it sets one tile at a zero-indexed coordinate (y, x) . In `line` mode, it draws a horizontal or vertical segment, either by an explicit endpoint or by a direction and length. In `rect` mode, it fills a rectangle or draws only its border, depending on the `filled` flag. The admissible tile names are generated dynamically from the current domain, so the same tool can place walls and empty cells in Binary, entities such as

player/key/door/enemy in Zelda, crates and targets in Sokoban, and domain-specific platformer tiles in Lode Runner or Super Mario Bros.

E.3 PCG Algorithms as Tools

This section gives the concrete PCG tools exposed to the agent. Binary generators internally use the convention `solid=0`, `empty=1`, while our level representation uses `EMPTY=0`, `WALL=1`; wrappers convert values before and after generation, check output shapes, compute changed cells, and apply the resulting edits through `EditManager`. Zelda wall-layout wrappers preserve player, key, door, and enemy tiles, applying Binary-style generation only to wall and empty cells. The generation tools generate or override a new level from scratch, while refinement tools modify the existing level in place.

Generation tools.

- `generate_random` ignores the current level and samples each Binary cell independently as wall with probability `solid_prob` and empty otherwise. The default is `solid_prob=0.5`. It is useful for introducing variation, but it gives no connectivity or path-length guarantee.
- `generate_maze` carves a perfect maze into the current wall tiles using randomized depth-first search. In simplified form, it chooses a starting wall cell, recursively visits neighboring cells in random order, and carves passages to unvisited cells. It only converts walls to empty cells, so an all-empty layout is unchanged.
- `generate_bsp` generates a Binary Space Partitioning room-and-corridor layout. It recursively splits the map into rectangular partitions, carves one room in each terminal partition, and links adjacent rooms with corridors. The defaults are `splits=3`, `min_width=5`, and `min_height=5`.
- `generate_digger` starts from an all-wall grid. A random walker carves empty tiles, occasionally carving square rooms, until the empty fraction reaches `stop_size`. The defaults are `change_prob=0.15`, `room_prob=0.01`, `room_size=3`, and `stop_size=0.3`.
- `generate_wfc` overwrites the current Binary layout using Wave Function Collapse from a built-in reference maze. It extracts local patterns of size `pattern_size=3` by default, then samples a layout consistent with those local constraints. If the constraint propagation fails, the tool returns failure and leaves the level unchanged.
- `generate_zelda_tile_placing` ignores the current Zelda layout and samples a complete Zelda map. It places random wall segments and enemies, then positions the player, key, and door on remaining empty cells. In expectation, roughly 20% of the cells are walls and roughly 5% are enemies; the required player, key, and door are always present, but reachability is not guaranteed.

Refinement tools.

- `generate_ca` smooths the current Binary layout in place using cellular automata. For `iterations=10` by default, each tile checks its 8-neighborhood; the default thresholds are `solid_count=2` and `empty_count=6`. The effect is to remove isolated tiles and fill small holes, producing smoother cave-like regions. Uniform all-empty or all-wall layouts are fixed points.
- `generate_connect` repairs Binary connectivity in place. It finds connected empty regions, fills regions smaller than `smallest_region_size=5` with walls, and connects the remaining regions with straight corridors. It is most useful after stochastic generators or local edits that create disconnected traversable components.

E.4 Evaluation, Rollback, and Acceptance

Tool execution is staged. At the start of a step, the optimizer checkpoints the current level, executes the LLM’s tool-call sequence, saves the resulting candidate level and tool results, and then rolls the edit back to the checkpoint. The candidate is evaluated separately with the PCG Benchmark-backed evaluator and scored by the optimization loss. Only after the accept/reject decision does the optimizer

install the candidate level as the new current level. This makes failed, low-scoring, or exploratory edits visible in the trace without allowing them to silently corrupt the accepted state.

The acceptance rule itself is summarized in Section 3.3. In the trace, rejected steps include the score change, metric deltas, tool-call outcomes, and a human-readable accept/reject reason, so the next LLM step can observe why the previous proposal failed.

F Additional Experiment Results

F.1 Full SMB Ablation Table

Table 7: Full SMB A* ablation and baseline results with Gemini 3 Flash on the quality task. This table expands Table 2 with benchmark quality, fixed exact-match counts, and pattern KL divergence.

Method	Quality $\uparrow$	Solvable	Target Align. $\uparrow$	Fixed Exact	Diversity $\uparrow$	Coherence $\uparrow$	Pat. JS $\downarrow$	Pat. KL $\downarrow$	Opt. Score $\uparrow$
Full agent (Our Method)	0.9965	30/30	0.7436	3/30	0.9333	0.8256	0.1753	0.87	83.17
No historical feedback	0.9967	30/30	0.7183	1/30	0.9000	0.8307	0.1770	0.94	81.67
No metric observation	0.9954	29/30	0.7638	3/30	0.9310	0.9108	0.1616	0.90	82.55
No evaluation loop	0.9867	21/30	0.6257	0/30	0.8571	0.9227	0.1586	0.93	14.66
Direct LLM one-shot	0.8896	25/30	0.3489	0/30	0.9200	0.8809	0.1487	0.69	—
Iterative direct LLM, 20 steps	0.9990	30/30	0.4644	0/30	1.0000	0.8664	0.1511	0.71	14.00
Random tool search	0.7766	9/30	0.1999	0/30	1.0000	0.7006	0.2090	1.58	-70.61
Evolutionary tool search	0.9907	29/30	0.9378	20/30	1.0000	0.6811	0.2317	3.77	89.03

G Structural Coherence Metric

The standard PCG Benchmark metrics used in the main experiments primarily measure functional properties: whether a level is playable, whether simulated agent behavior matches target values, and whether generated levels differ from one another under task-specific behavioral or structural distances. These quantities are necessary for controlled evaluation, but they do not fully capture whether a level appears intentionally designed. In particular, a search procedure can satisfy SMB target metrics by placing many isolated tiles or objects in visually irregular patterns. We therefore introduce a lightweight *structural coherence* metric as an auxiliary, automatically computed proxy for qualitative level organization.

The metric is not intended to replace human judgments of fun, challenge, or aesthetic quality. Instead, it measures a narrower property: whether the tile layout has local continuity, moderate object density, and spatial organization consistent with recognizable platformer structure. We use it in the SMB ablation analysis to distinguish levels that satisfy functional objectives from levels that also appear less fragmented under a hand-specified structural prior.

Domain-agnostic local coherence. For a tile map $L \in \mathcal{T}^{H \times W}$, we first compute three local anti-fragmentation statistics. The transition density r_{trans} is the fraction of horizontally or vertically adjacent tile pairs whose tile types differ. The singleton ratio r_{single} is the fraction of cells with no four-neighbor cell of the same tile type. The small-component ratio r_{comp} is the fraction of cells belonging to a four-connected same-tile component of size at most two. Each statistic is converted to a score in $[0, 1]$ using a linear penalty that is 1 below a “good” threshold and 0 above a “bad” threshold:

$$g_{\text{low}}(x; a, b) = \text{clip}_{[0,1]} \left(1 - \frac{x - a}{b - a} \right). \quad (1)$$

The local coherence score is then

$$C_{\text{local}} = 0.45 g_{\text{low}}(r_{\text{trans}}; 0.18, 0.55) + 0.30 g_{\text{low}}(r_{\text{single}}; 0.02, 0.20) + 0.25 g_{\text{low}}(r_{\text{comp}}; 0.03, 0.18). \quad (2)$$

For non-SMB domains, this local score is the reported coherence score. It favors contiguous regions and penalizes salt-and-pepper tile noise without assuming domain-specific semantics.

SMB design coherence. For Super Mario Bros., local smoothness alone is insufficient: a flat empty level can be locally coherent while still being uninteresting, and a functional but noisy level can exploit the simulator while looking unnatural. We therefore combine local coherence with five SMB-specific design terms. Let C_{profile} measure whether the inferred ground surface changes smoothly

between adjacent columns, counting neighboring ground columns as smooth when their vertical positions differ by at most two tiles. Let C_{corridor} reward a moderate amount of solid structure in the playable corridor, penalizing corridors that are either nearly empty or frequently blocked. Let C_{density} reward a moderate ratio of interactive objects, including bricks, question blocks, tubes, coins, and enemies. Let C_{pacing} reward these objects being distributed across horizontal segments rather than concentrated in one region. Finally, let C_{clutter} directly penalize high transition density using a stricter SMB-specific threshold.

The SMB coherence score is the weighted sum

$$C_{\text{SMB}} = 0.18 C_{\text{profile}} + 0.18 C_{\text{corridor}} + 0.20 C_{\text{density}} + 0.10 C_{\text{pacing}} + 0.20 C_{\text{clutter}} + 0.14 C_{\text{local}}. \quad (3)$$

All component scores lie in $[0, 1]$, so $C_{\text{SMB}} \in [0, 1]$. We deliberately exclude two quantities that are already covered by the underlying PCG Benchmark quality checks or were too weakly aligned with the qualitative issue of interest: enemy support and total ground coverage. These values can still be logged as diagnostics, but they do not contribute to the reported coherence score.

Use in ablation reporting. For Tables 2 and 7, we recompute structural coherence from each final SMB level and report the average across 30 trials. This post-hoc score helps identify reward-hacking behavior in which a method achieves high functional quality while producing fragmented layouts. In the SMB ablation package, Evolutionary Tool Search achieves strong target alignment and exact target matching, but has lower structural coherence than the full agent, matching qualitative inspection of its generated levels.

H Pattern Divergence Metrics

To compare whether generated SMB levels preserve local design statistics from human-authored Mario content, we additionally compute tile-pattern divergence metrics following the general approach of tile-pattern KL divergence for level analysis [42]. These metrics are post-hoc diagnostics only; they are not included in the optimization score and are not shown to the agent during generation.

Reference corpus. The SMB problem in PCG Benchmark represents Mario levels through vertical slices sampled from original Super Mario Bros. levels, following the n-gram slice representation used in prior Mario level-generation work [10, 2]. We use the benchmark’s local slice reference file, `pcg_benchmark/probs/smb/slices.txt`, as the human-designed reference corpus. In this workspace, that file contains 126 vertical slices of height 16. We concatenate these slices horizontally into a reference tile array $R \in \mathcal{T}^{16 \times 126}$. This is a local-style reference rather than a full ordered reconstruction of the original levels: it preserves human-designed tile neighborhoods present in the benchmark slice vocabulary, but it does not claim to preserve the complete long-range ordering of the source levels.

Pattern distributions. For a tile array $L \in \mathcal{T}^{H \times W}$ and a window size $k = 2$, let $\mathcal{P}_k(L)$ be the multiset of all overlapping $k \times k$ tile patches in L . Each patch is flattened into a discrete pattern id $p \in \mathcal{T}^{k^2}$. We compute the empirical distribution

$$P_L(p) = \frac{\sum_{q \in \mathcal{P}_k(L)} \mathbf{1}[q = p]}{|\mathcal{P}_k(L)|}. \quad (4)$$

For a set of generated levels $\mathcal{G}$, we compute per-level distributions P_L for each $L \in \mathcal{G}$ and average the resulting divergence scores in Tables 2 and 7. We also compute corpus-level diagnostics in our experiment summaries by pooling pattern counts across all generated levels before normalizing, but the tables report the per-level average because it matches the trial-wise reporting of the other columns.

KL and JS divergence. Let P_R be the reference pattern distribution and P_L be the generated-level distribution. Because KL divergence is undefined when $P_L(p) = 0$ for a pattern with $P_R(p) > 0$, we apply additive smoothing over the union of observed patterns:

$$\tilde{P}(p) = \frac{P(p) + \epsilon}{\sum_{p' \in \mathcal{U}} (P(p') + \epsilon)}, \quad \mathcal{U} = \text{supp}(P_R) \cup \text{supp}(P_L), \quad \epsilon = 10^{-8}. \quad (5)$$

The reported pattern KL is generated-to-reference KL,

$$D_{\text{KL}}(\tilde{P}_L \parallel \tilde{P}_R) = \sum_{p \in \mathcal{U}} \tilde{P}_L(p) \log \frac{\tilde{P}_L(p)}{\tilde{P}_R(p)}. \quad (6)$$

This direction penalizes local tile patterns that appear frequently in generated levels but are rare or absent in the human-designed reference corpus. We also report Jensen–Shannon divergence,

$$D_{\text{JS}}(\tilde{P}_L, \tilde{P}_R) = \frac{1}{2} D_{\text{KL}}(\tilde{P}_L \parallel M) + \frac{1}{2} D_{\text{KL}}(\tilde{P}_R \parallel M), \quad M = \frac{1}{2}(\tilde{P}_L + \tilde{P}_R), \quad (7)$$

which is symmetric and more stable as a compact similarity diagnostic. Lower values of both metrics indicate closer agreement with the local tile-pattern statistics of the reference corpus. These scores should not be interpreted as human preference judgments: they capture local style similarity only, and therefore complement rather than replace functional metrics, structural coherence, and qualitative inspection.

I Environment Details

I.1 Binary Maze

The binary maze problem requires the agent to produce a two-dimensional grid of size $16(\text{height}) \times 16(\text{width})$, containing exactly two tile types: empty and wall. An agent may traverse empty tiles using four-directional movement (up, down, left, and right). For this problem, we focus on 2 main metrics: longest shortest path between any two pairs of traversable points, and numbers of connected region. We calculate the longest shortest path and numbers of regions using Dijkstra algorithm and flood filling algorithm, respectively, the process is same as [27].

- **Quality:** For quality metric in pcg benchmark, a well-formed maze should contain a single connected component of empty tiles, ensuring that every passable cell is reachable from every other. Also the longest shortest path should exceed half of the maximum possible path length. To simplify the calculation process, we use the length of pure zig-zag that fill the whole map as an approximate of the maximum possible path in the map.
- **Controllability:** the controllability score of binary centers the plateau on the sampled control value between add/subtract error. In this problem we only measure the controllability of the path length and the acceptable error is 10% of the sampled target.
- **Diversity:** the diversity score for binary maze measures Hamming distance of all the grids between two levels.

I.2 Binary Maze with Doors

For the binary maze setting, maximizing the longest possible path tends to produce highly degenerate solutions. On a fixed-size grid, the optimal construction is essentially limited to a small set of near-perfect zigzag patterns, resulting in low structural diversity. To alleviate this issue, we introduce a binary-with-doors variant, inspired by prior work [26]: two boundary doors (entrance/exit cells) are sampled randomly according to a predefined procedure 1. Compared to the vanilla binary setting, the LLM agent not only needs to construct a maze that meets the target metrics, but also ensure that the two sampled doors are connected. Because the door locations are random, the true longest path connecting an arbitrary pair of doors is closely related to finding a Hamiltonian path, which is NP-complete in general. Consequently, in our metric computation we approximate the maximum achievable path length using the length of the longest zigzag pattern on the same board size, and use this value to normalize/score path-length performance.

- **Quality:** The quality score is the arithmetic mean of two trapezoidal reward components:
- **Controllability:** The controllability score again measures how precisely the agent can steer connected path to a specified target value. Only solvable trials (where connected path > 0) contribute to the aggregate controllability score; unsolvable trials receive a score of 0
- **Diversity:** Diversity between two levels is measured via normalized Hamming distance over their flattened binary representations. This is computed pairwise across all solvable trials (where connected path > 0) that have connected path in a diversity experiment.

Algorithm 1 Random Door Placement on the Augmented Grid Perimeter. Design rationale: seeded per `door_seed` for reproducibility; enforce minimum perimeter separation `peri_dist` $\geq \min(W, H)$ to avoid trivial adjacent doors.

Require: Augmented grid width W , height H ; RNG seed s (`door_seed`, default 42)

Ensure: Two door coordinates `door1`, `door2` on non-corner perimeter cells

```

1: Build perimeter cycle  $P$  (clockwise, excluding corners).
2:  $P \leftarrow []$ 
3: Append top row left→right (excluding corners) to  $P$ 
4: Append right column top→bottom (excluding corners) to  $P$ 
5: Append bottom row right→left (excluding corners) to  $P$ 
6: Append left column bottom→top (excluding corners) to  $P$ 
7:  $L \leftarrow |P|$ 

8: Pick the first door (seeded).
9: Initialize RNG with seed  $s$ 
10:  $\text{idx1} \leftarrow \text{randint}(0, L - 1)$ 
11:  $\text{door1} \leftarrow P[\text{idx1}]$ 

12: Find valid candidates for the second door.
13:  $d_{\min} \leftarrow \min(W, H)$ 
14:  $C \leftarrow []$ 
15: for  $i = 0$  to  $L - 1$  do
16:   if  $i = \text{idx1}$  then
17:     continue
18:   end if
19:    $d_{\text{fwd}} \leftarrow (i - \text{idx1}) \bmod L$ 
20:    $d_{\text{bwd}} \leftarrow (\text{idx1} - i) \bmod L$ 
21:    $\text{peri\_dist} \leftarrow \min(d_{\text{fwd}}, d_{\text{bwd}})$ 
22:   if  $\text{peri\_dist} \geq d_{\min}$  then
23:     Append  $i$  to  $C$ 
24:   end if
25: end for

26: Pick the second door (with fallback).
27: if  $|C| > 0$  then
28:    $\text{idx2} \leftarrow \text{choice}(C)$ 
29: else
30:    $\text{idx2} \leftarrow \arg \max_{i \in \{0, \dots, L-1\} \setminus \{\text{idx1}\}} \min((i - \text{idx1}) \bmod L, (\text{idx1} - i) \bmod L)$ 
31: end if
32:  $\text{door2} \leftarrow P[\text{idx2}]$ 
33: return (door1, door2)

```

I.3 Zelda

The Zelda dungeon problem requires the agent to produce a top-down two-dimensional grid based game of size $16(\text{height}) \times 16(\text{width})$, containing six tile types: wall, empty, player, key, door, and enemy. The player needs to navigate to collect a key, then reach an exit door, while killing the enemies in the way. For this problem, we focus on 4 main metrics: shortest path length from player to key (player key), shortest path length from key to door (key door), number of connected regions, and entity counts (players, keys, doors, enemies). We calculate the shortest paths using Dijkstra’s algorithm, treating enemy tiles as traversable tiles. The number of connected regions is computed via flood fill.

- **Quality:** A well-formed Zelda level must contain exactly one player, one key, and one door, with a single connected region of passable tiles. The level must be playable, meaning a path exists from the player to the key and from the key to the door. Additionally, the combined solution length (player key + key door) should be close to a target length of $W + H$

(32 for a 16×16 level), and the enemy count should be close to its target value. The quality score averages four sub-scores: region correctness, entity count accuracy, and playability (which itself includes a solution length reward when both paths exist).

- **Controllability:** The controllability score measures how precisely the agent can achieve sampled target values for `player key` and `key door` independently. The acceptable error is 10% of each sampled target value. The final controllability score is the average of the two path length scores. Only solvable levels (both paths exist) contribute.
- **Diversity:** The diversity score compares the solution paths of two levels. Each level’s path is the concatenation of the player-to-key and key-to-door coordinate sequences. Before comparison, paths are normalized by mirroring coordinates when the starting point falls in the right or bottom half of the grid to account for spatial symmetry. The similarity between two normalized paths is computed using Python’s `SequenceMatcher`, which finds the longest common subsequences and returns a ratio in $[0, 1]$. The diversity score reaches 1.0 when the path dissimilarity ($1 - \text{ratio}$) exceeds 0.3, and ramps linearly from 0 to 1 for smaller differences.

I.4 Lode runner

Lode Runner is a two dimensional $11(\text{height}) \times 16(\text{width})$ side-view platformer level design task with seven tile types: solid, empty, player, gold, enemy, ladder, and rope. The player may walk on supported empty tiles, climb ladders, move along ropes, and fall when unsupported. Rather than simulating gameplay in real time, we evaluate levels with a static BFS reachability analysis over states $(x, y, \text{falling})$ starting from the player position. The resulting reachability map determines whether all gold is collectable and how much of the structural layout is traversed. Because gold and enemies are treated as static markers, the problem reduces to constrained reachability. A valid playable level must contain exactly one player, at least one gold tile, and permit collection of all gold. For this problem, we focus on two controllable metrics: the number of ladder tiles and the number of rope tiles.

- **Quality:** The PCG benchmark evaluates quality through four progressive tiers, each gated on the previous tier achieving a score of ≥ 1.0 . Tier 1 (Stats) checks that entity counts are reasonable: exactly 1 player, gold count near a target, and enemy count near a target. Tier 2 (Exploration) requires that at least 20% of the level is reachable by the player via BFS exploration. Tier 3 (Playability) requires that all gold is collectible and that at least 75% of structural tiles (ladders, ropes, floor supports) are actually traversed by the player. Tier 4 (Decoration) measures how well the level’s movement pattern distributions (walking, hanging, climbing, falling) match reference levels from the original game using Jensen–Shannon divergence, and penalizes excessive disconnected structural regions. The final quality score is the average of all four tiers, each in $[0, 1]$.
- **Controllability:** The controllability score measures how closely the generated level matches sampled target counts for ladder and rope tiles. Each metric is scored again using a trapezoidal reward function centered on the target value with an acceptable error of 2% of the total grid area. The final controllability score is the average of the ladder and rope scores. Only playable levels (all gold reachable by the player) contribute to the controllability score and unplayable levels receive a score of 0.
- **Diversity:** The diversity score compares the exploration maps of two levels by computing the element-wise difference across four movement-type layers (walking, climbing, hanging, and falling). These differences are combined with weights of 0.3 each for walking, climbing, and hanging, and 0.1 for falling. The weighted sum is scored against a target of 40% of the total grid area differing between the two levels. Only playable levels are included in the diversity computation.

I.5 Sokoban

The Sokoban problem requires the agent to produce a two-dimensional grid based puzzle game level of size $8(\text{height}) \times 8(\text{width})$, containing five tile types: solid (wall), empty (floor), player, crate, and target. The player can move in four directions (up, down, left, and right). When the player moves into a crate, the crate is pushed one tile in that direction, and crates cannot be pushed through walls or other crates or be pulled. The puzzle is solved when all crates are placed on target locations. A valid

level must contain exactly one player and an equal number of crates and targets. For this problem, the main metrics are: the number of crates, solvability status, and solution length (number of moves to solve).

Solvability is determined by a cascade of four solvers, each limited to 5,000 node expansions: BFS first, then A* with decreasing cost weights ($\text{balance} \in \{1.0, 0.5, 0.0\}$), where the priority function is $f = h + \text{balance} \cdot g$, where h is the sum of Manhattan distances from each crate to its nearest unmatched target (greedy one-to-one assignment), and g is the search depth. A level is solvable if any of the four solvers reaches a winning state within its budget. During search, states where a crate is pushed into a deadlock position (e.g., a corner not on a target) are pruned. Different balance values explore the search space differently: BFS is optimal but exhausts its budget quickly on complex puzzles, while pure greedy ($\text{balance} = 0$) aggressively chases low-heuristic states and can find solutions that BFS misses.

- **Quality:** The quality metric combines two components averaged equally: (1) a structural score, and (2) a solver score. The structural score is the mean of three sub-rewards that each range from 0 to 1: player count (peaks at exactly 1), crate count (peaks at 1 or more), and crate-target balance (peaks when the difference is 0). Each sub-reward increases linearly from 0 at the boundary to 1 at the target plateau. The solver score is only computed when the solver successfully runs (i.e., the level has valid structure); otherwise it is 0. It is the mean of two sub-rewards: a heuristic reward that peaks when $h = 0$ (all crates on targets) and decays linearly as h increases, and a solution length reward that peaks when the solution is at least $(W + H) \times \text{difficulty}$ moves long, where difficulty is a per-variant constant set to 1 for the standard 8×8 variant.
- **Controllability:** The controllability score of Sokoban measures how closely the generated level matches a sampled target crate count. The reward plateaus at full score when the actual crate count is within ± 1 of the target, and decreases linearly outside that range. Only solvable levels (solution length > 0) contribute to the controllability score; unsolvable levels receive a score of 0.
- **Diversity:** The diversity score for Sokoban measures pairwise dissimilarity of solutions between levels. Each solution is first canonicalized (flipped to a consistent orientation) and then compressed by collapsing consecutive identical moves into a single character (e.g., “rrruuuddd” $\rightarrow$ “rud”). Pairwise similarity is computed using `SequenceMatcher`, and the diversity reward requires at least 50% difference for full score. Only solvable levels are included in pairwise comparisons.

I.6 Super Mario Bros

The Super Mario Bros (SMB) problem requires the agent to produce a two-dimensional grid of side-view platformer game of size $16(\text{height}) \times 32(\text{width})$, containing ten tile types: empty, solid, ladder, brick, question block, tube, coin, goomba, koopa, and spiny. Unlike other problems where metrics are computed from the level layout alone, SMB metrics are gameplay simulation outcomes obtained by running a Mario AI agent through a physics simulation of the generated level. The simulation uses a pure-Python reimplement of the Mario AI Framework with full physics, sprites movements, and collision. The engine uses a scrolling camera of size 256×256 pixels (16×16 tiles), centered on Mario and clamped to the level bounds. Enemies are only spawned and begin moving once the camera reaches their tile position; until then they remain frozen at their spawn locations. Sprites that move more than 64 pixels beyond the camera horizontally, or fall below the level, are despawned.

Before simulation, three static pre-checks must pass: the empty tile ratio must exceed 0.5, tube tiles must be properly paired per row, and the ratio of enemies not standing on solid ground must be below 0.1. If any pre-check fails, the simulation is skipped entirely and all gameplay metrics default to zero.

When the pre-checks pass, a Mario AI agent plays the level to produce gameplay metrics. We evaluate under two solver configurations: *auto*, where a heuristic agent (always moves right and jumps) attempts the level first and falls back to A* search if it fails to complete; and *astar*, where the A* agent is used directly. The A* agent uses heuristic $h = (x + 10 \cdot v_x - x_{\text{root}}) - 10^6 \cdot \text{damage}$, where x is Mario’s current pixel position, v_x is his horizontal velocity, and x_{root} is his position at the start of the search. The velocity term biases the search toward states with rightward momentum, while the large

damage penalty causes the agent to actively avoid enemies rather than engage them. The simulation produces four key gameplay metrics: completion percentage, enemies killed (excluding enemies that fall off ledges on their own), coins collected, and voluntary ground jumps (stomp bounces do not count).

- **Quality:** The quality score combines structural validity and gameplay completion:

$$q = \frac{q_{\text{tube}} + q_{\text{empty}} + q_{\text{noise}} + q_{\text{fenemies}} + 4 \cdot \text{complete}}{8} \quad (8)$$

where q_{tube} rewards zero tube layout issues, q_{empty} rewards an empty tile ratio above 0.5, q_{noise} penalizes horizontal tile transition noise relative to a reference profile (only computed when $q_{\text{tube}} = 1$ and $q_{\text{empty}} = 1$), q_{fenemies} rewards zero floating enemies, and $\text{complete} \in [0, 1]$ is the fraction of the level traversed by the gameplay simulation AI agent. Each component uses a trapezoidal reward function in $[0, 1]$. Completion is weighted $4\times$ more than any structural component, emphasizing playability.

- **Controllability:** The controllability score measures whether the gameplay agent’s simulation outcomes match sampled target values for three metrics: enemies killed, jumps, and coins collected:

$$c = \frac{c_{\text{enemies}} + c_{\text{jumps}} + c_{\text{coins}}}{3} \quad (9)$$

Each component uses a trapezoidal reward centered on the target value with an error margin of $\lfloor 0.1 \cdot v_{\text{max}} \rfloor$, where v_{max} is the maximum of the control range (4 for enemies, 6 for jumps, 3 for coins at width 32). But since all error margins evaluate to 0 for the map size of width 32, exact matches are required for a perfect score, making controllability particularly challenging. Only trials where the gameplay agent completes the level ($\text{complete} \geq 1.0$) contribute to the controllability score.

- **Diversity:** The diversity score compares the gameplay agent’s trajectory heatmaps between two levels rather than the level grids themselves. Per-frame pixel positions of Mario are converted to tile-grid visitation counts on a 16×32 heatmap matching the level dimensions. The L_1 distance between the two heatmaps, normalized by total frame count, is then passed through a trapezoidal reward with plateau at 0.4:

$$d = f_{\text{reward}}\left(\frac{\sum |V_1 - V_2|}{\sum V_1 + \sum V_2}, 0, 0.4, 1.0\right) \quad (10)$$

where V_1, V_2 are the visitation heatmaps. This measures whether different levels produce meaningfully different play experiences, not just different tile arrangements.

J More Qualitative Results

Binary

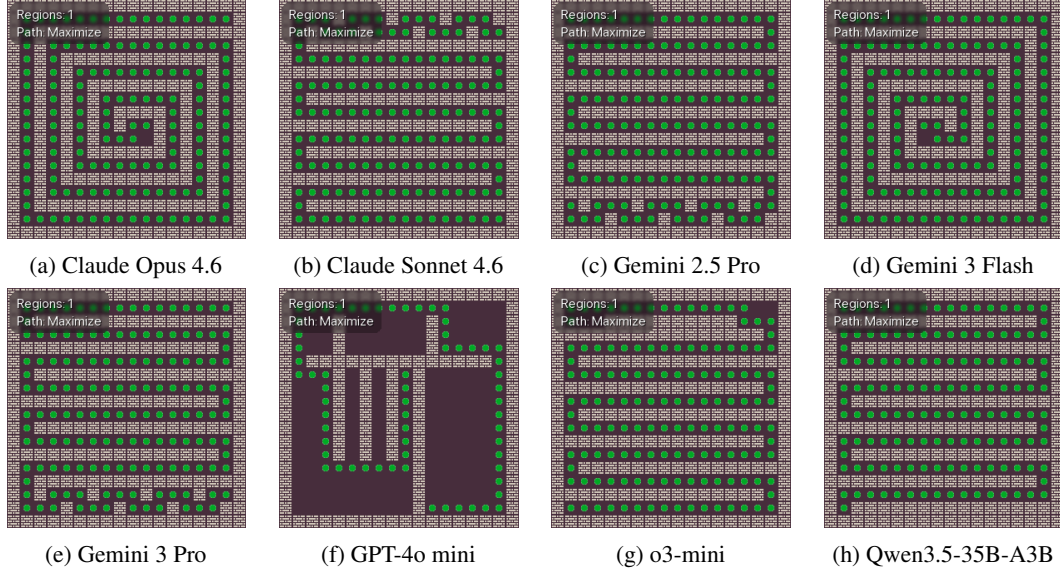


Figure 11: Qualitative comparison of levels generated by different backbone models in the binary problem experiment under the same target metrics.

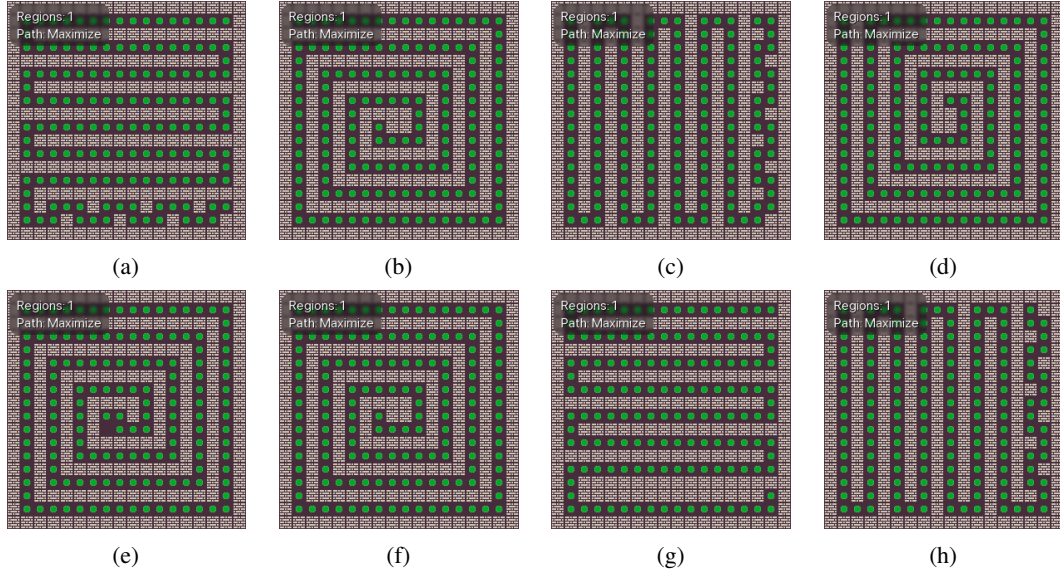


Figure 12: Qualitative diversity of levels generated by Gemini 3 Pro in the binary problem experiment under the same target metrics.

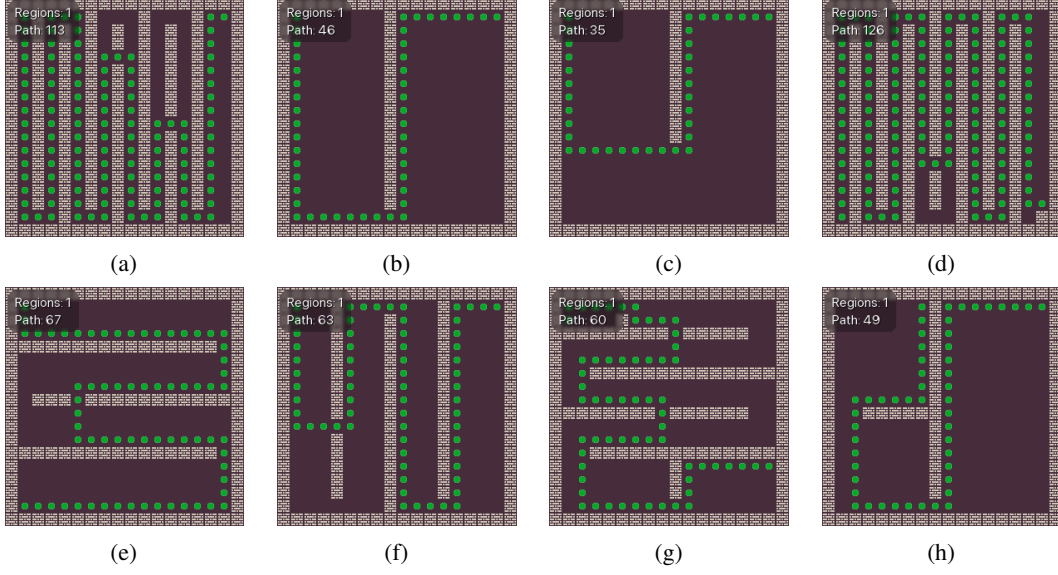


Figure 13: Qualitative controllability examples generated by Gemini 2.5 Pro in the binary problem experiment under different target metrics.

Binary Door

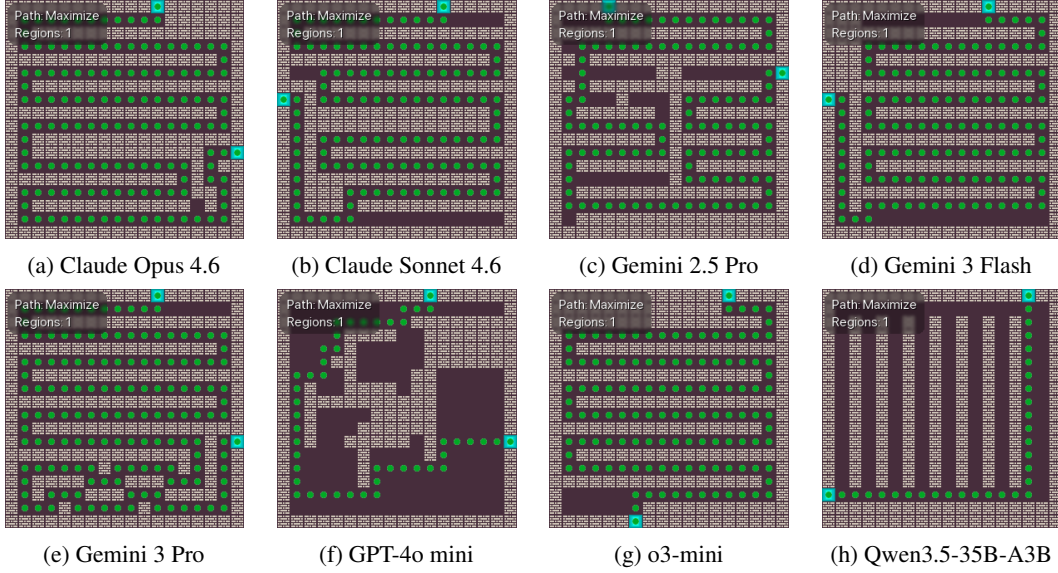


Figure 14: Qualitative comparison of levels generated by different backbone models in the binary door problem experiment under the same target metrics.

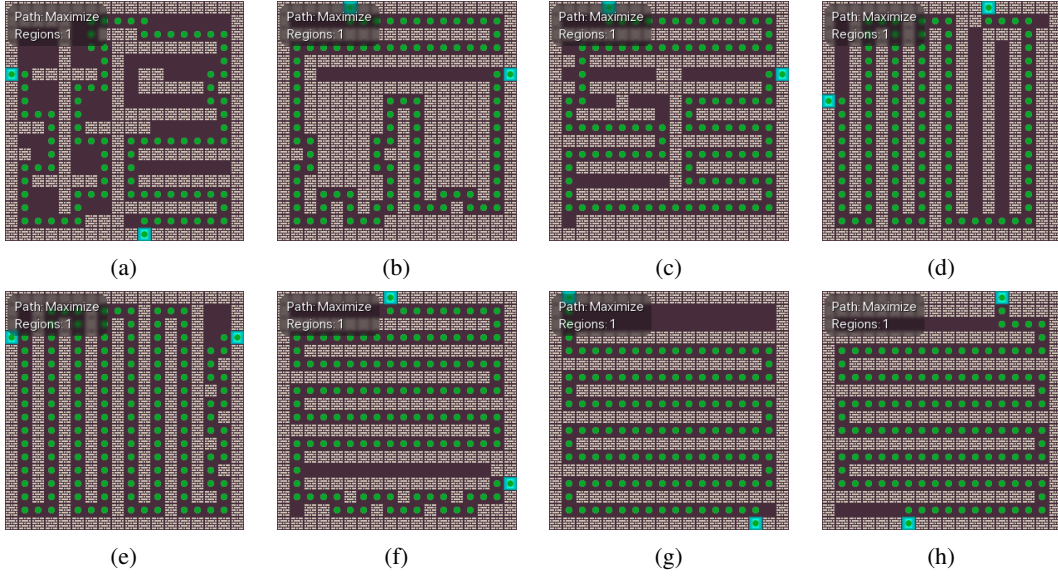


Figure 15: Qualitative diversity of levels generated by Gemini 2.5 Pro in the binary door problem experiment under the same target metrics.

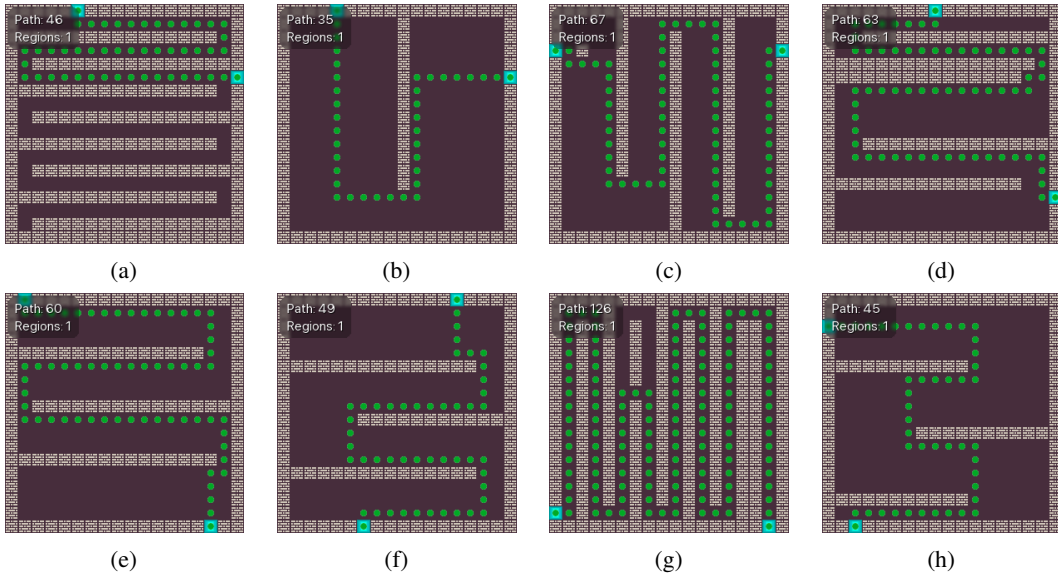


Figure 16: Qualitative controllability examples generated by Gemini 2.5 Pro in the binary door problem experiment under different target metrics.

Lode Runner

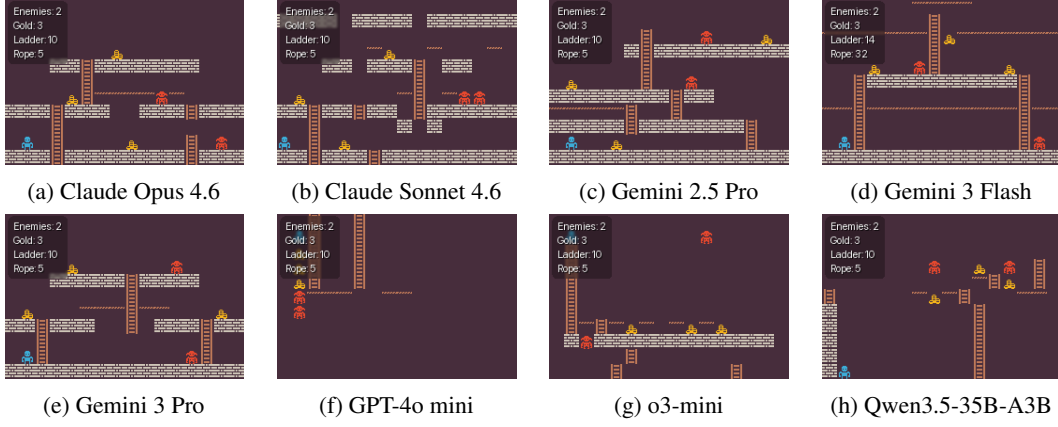


Figure 17: Qualitative comparison of levels generated by different backbone models in the Lode Runner problem experiment under the same target metrics.

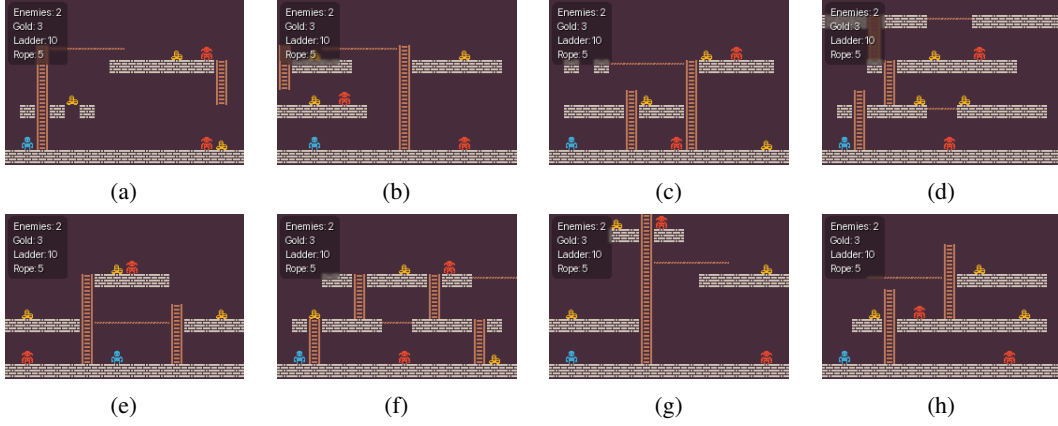


Figure 18: Qualitative diversity of levels generated by Gemini 3 Pro in the Lode Runner problem experiment under the same target metrics.

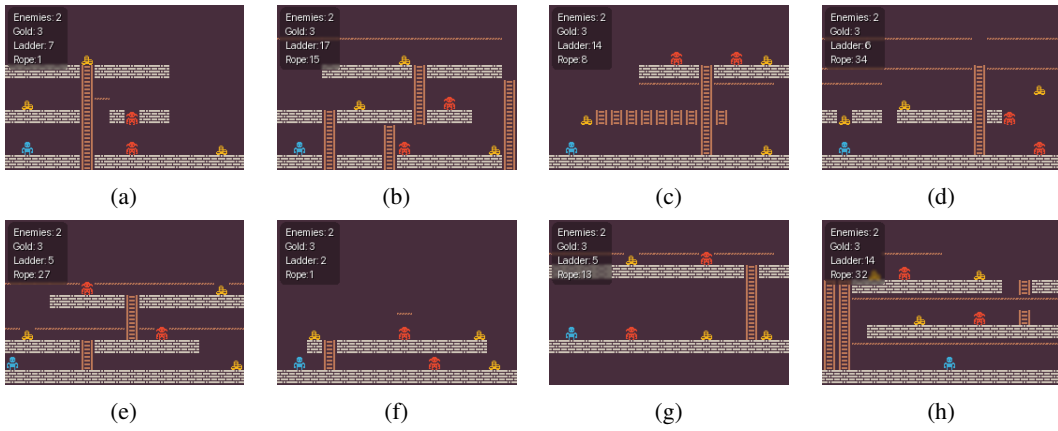
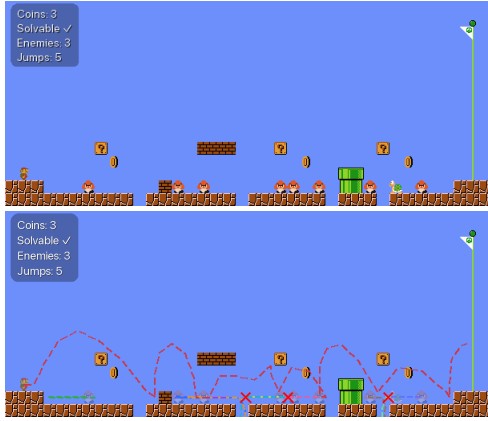


Figure 19: Qualitative controllability examples generated by Gemini 2.5 Pro in the Lode Runner problem experiment under different target metrics.

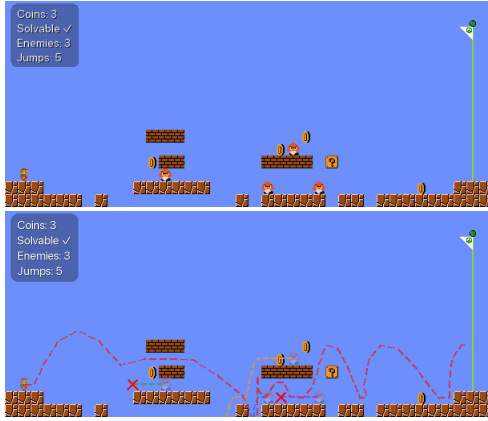
Super Mario Bros (A*)



(a) Claude Opus 4.6



(b) Claude Sonnet 4.6

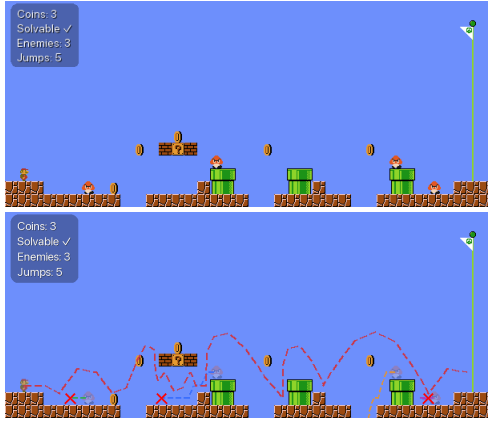


(c) Gemini 2.5 Pro

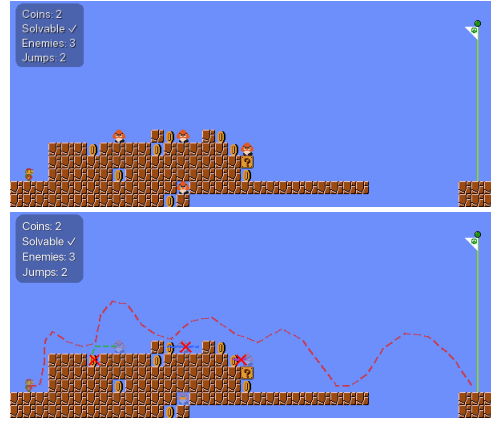


(d) Gemini 3 Flash

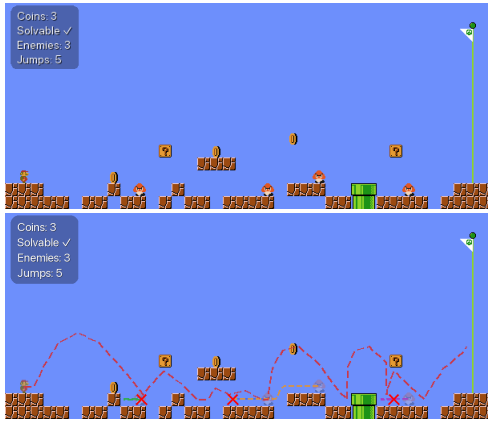
Figure 20: Qualitative comparison of levels generated by different backbone models in the SMB (A*) problem experiment under the same target metrics (1/2). Top: level layout. Bottom: agent trajectory.



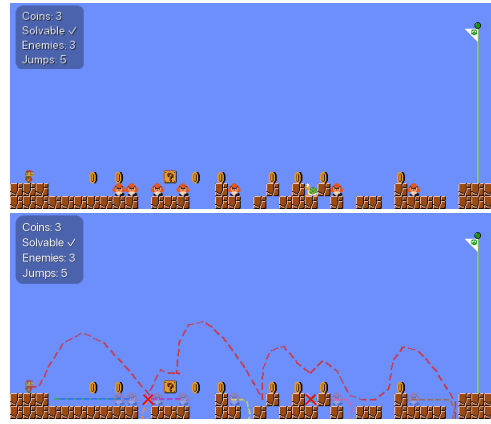
(a) Gemini 3 Pro



(b) GPT-4o mini



(c) o3-mini



(d) Qwen3.5-35B-A3B

Figure 21: Qualitative comparison of levels generated by different backbone models in the SMB (A*) problem experiment under the same target metrics (2/2).

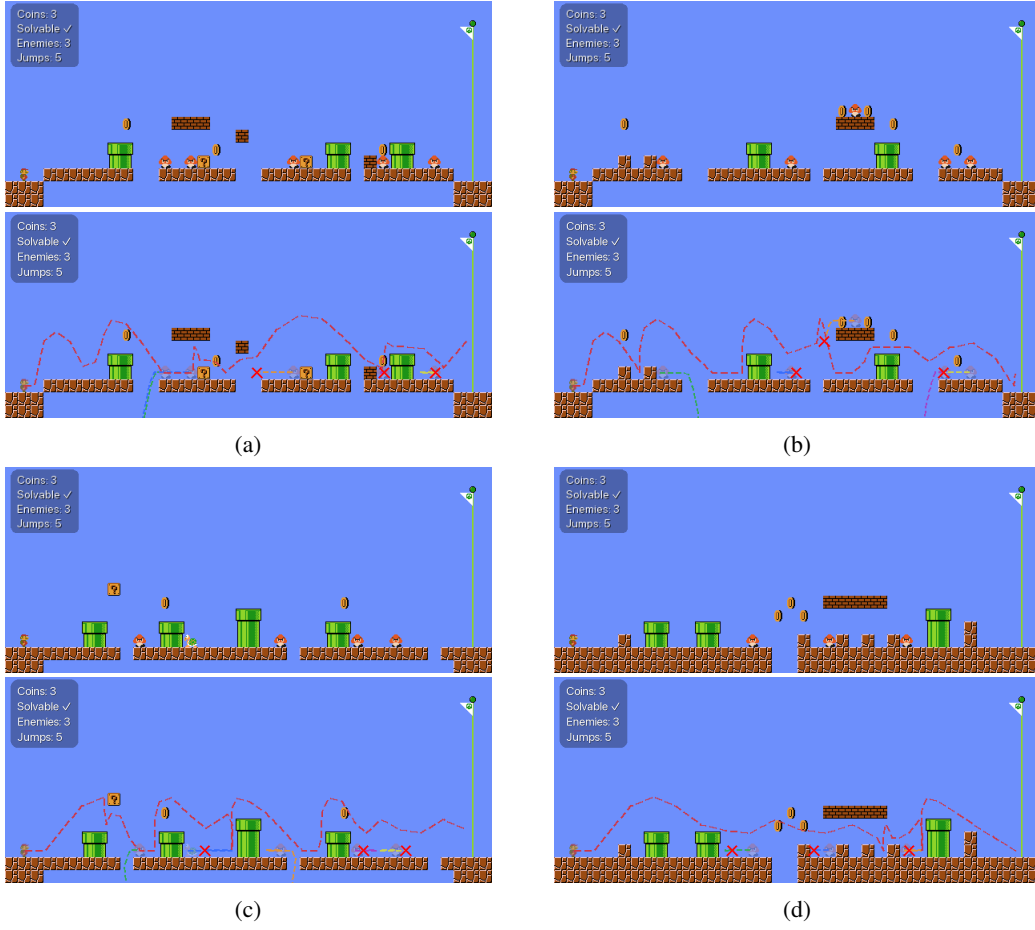


Figure 22: Qualitative diversity of levels generated by Gemini 3 Pro in the SMB (A*) problem experiment (1/2). Top: level layout. Bottom: agent trajectory.

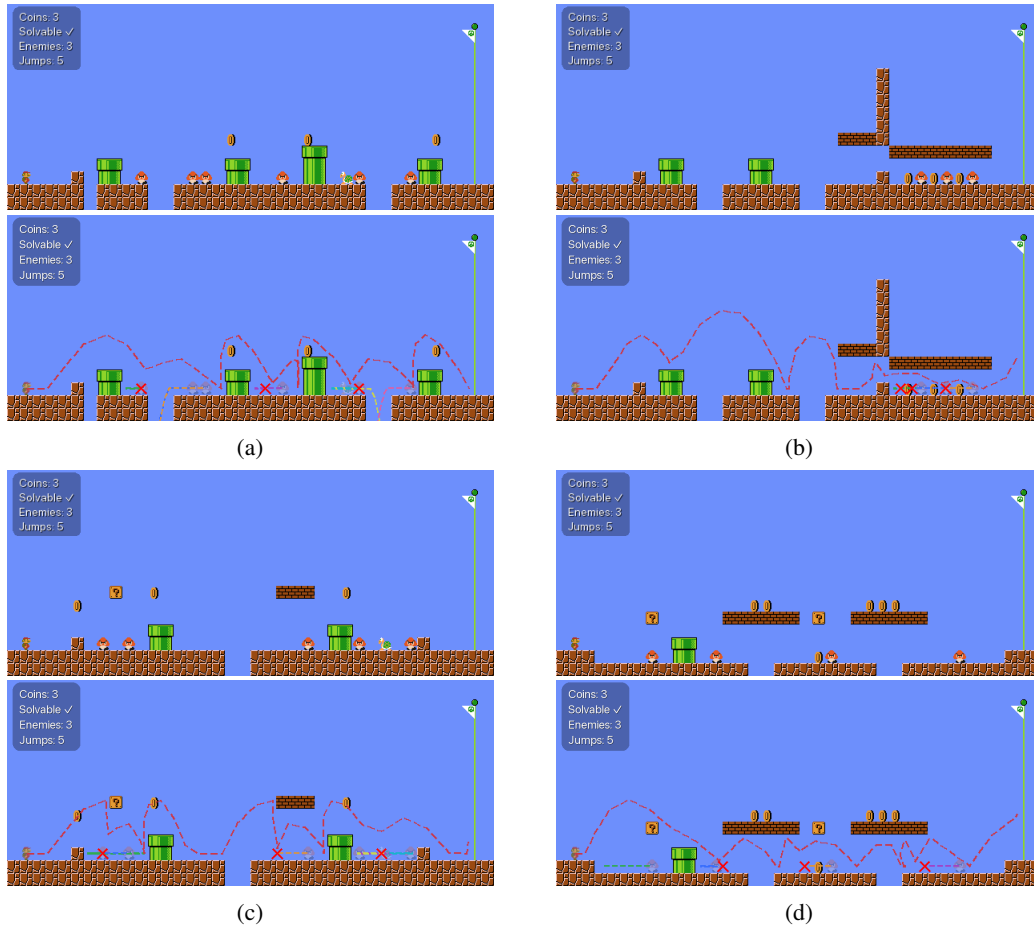


Figure 23: Qualitative diversity of levels generated by Gemini 3 Pro in the SMB (A*) problem experiment (2/2).

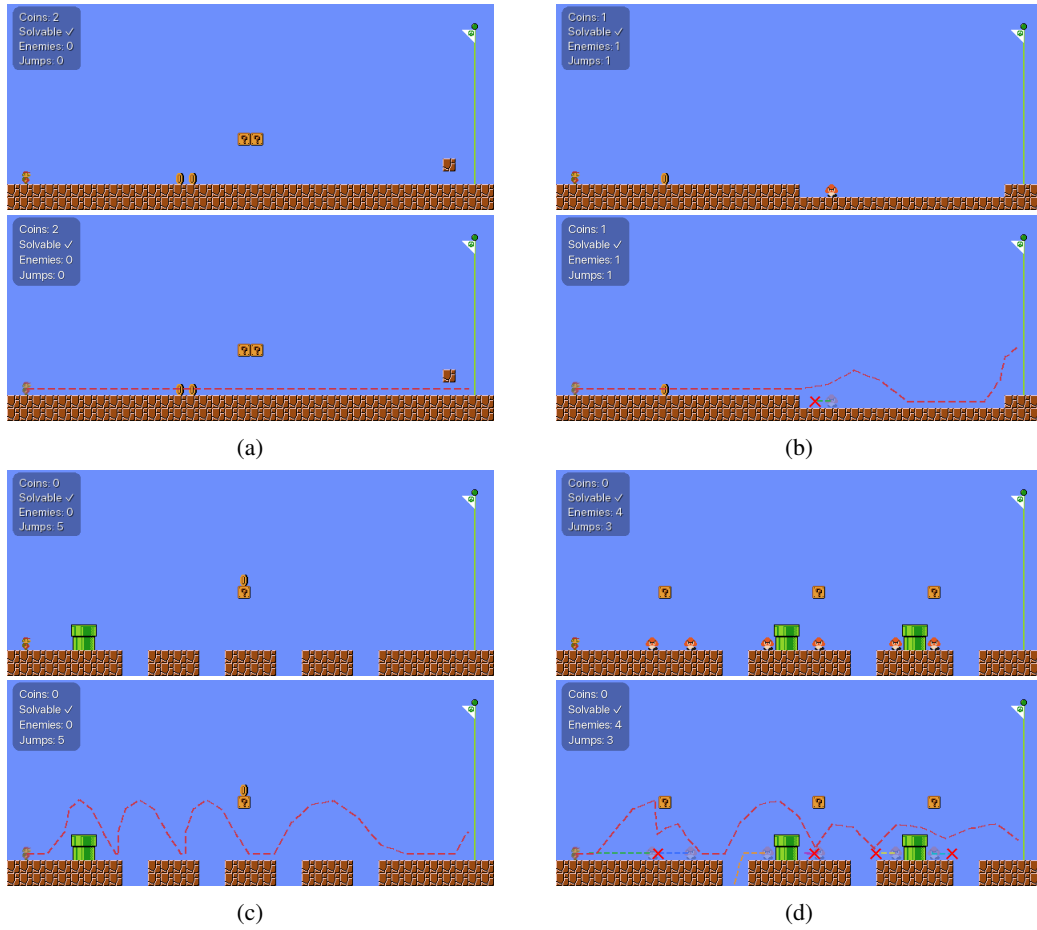


Figure 24: Qualitative controllability examples generated by Gemini 3 Flash in the SMB (A*) problem experiment under different target metrics (1/2). Top: level layout. Bottom: agent trajectory.

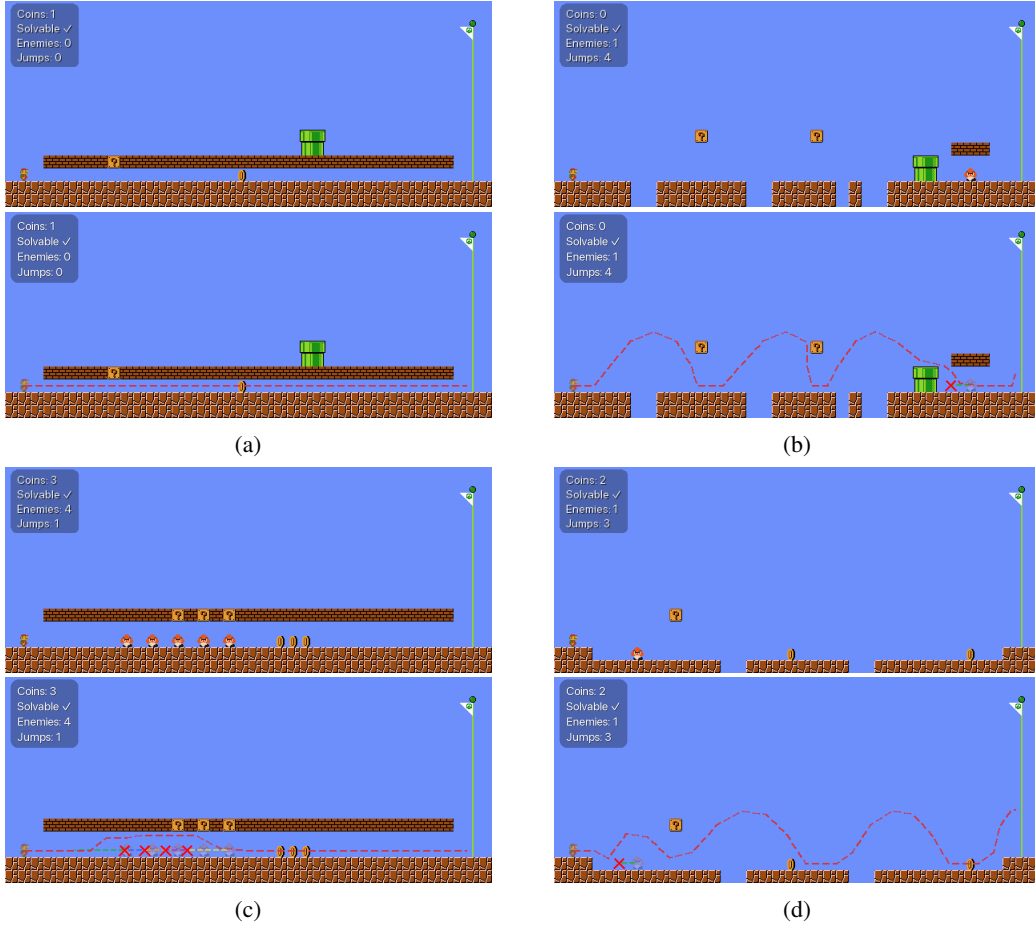


Figure 25: Qualitative controllability examples generated by Gemini 3 Flash in the SMB (A*) problem experiment under different target metrics (2/2).

Super Mario Bros (A*) – Evolutionary Tool Search in the Ablation

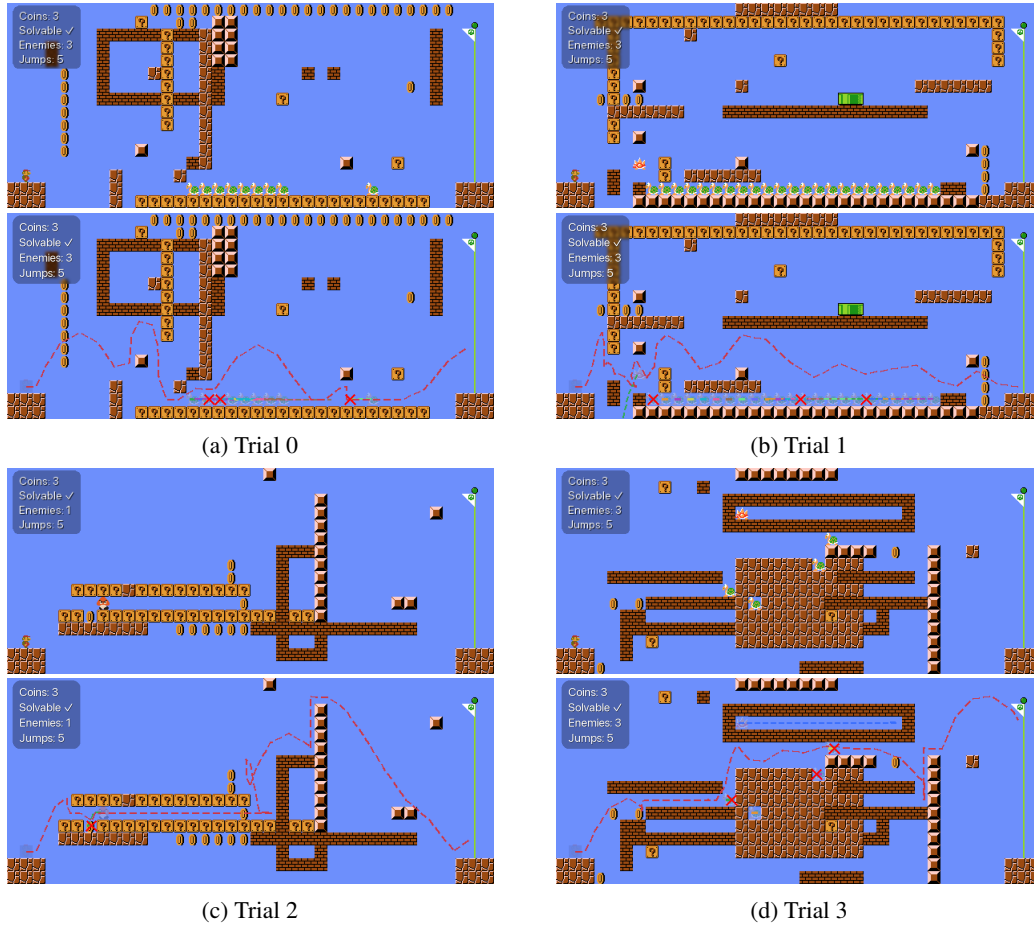
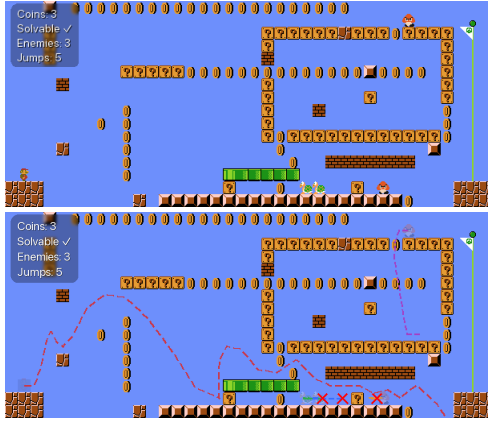
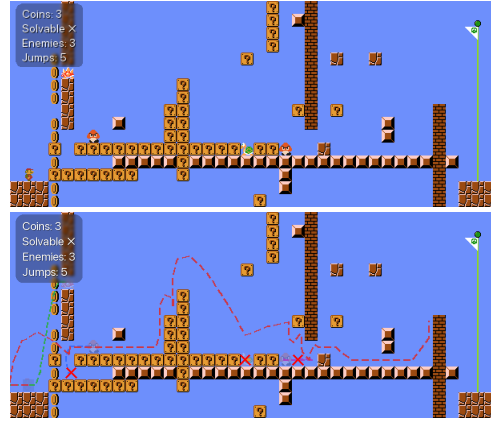


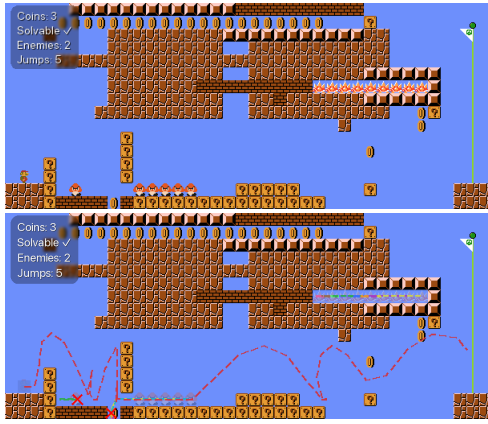
Figure 26: Qualitative examples from the SMB (A*) Evolutionary Tool Search ablation (1/8). Top: level layout. Bottom: agent trajectory. Tags show final gameplay outcomes.



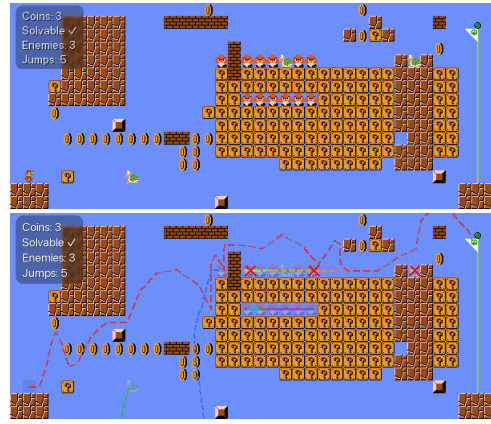
(a) Trial 4



(b) Trial 5

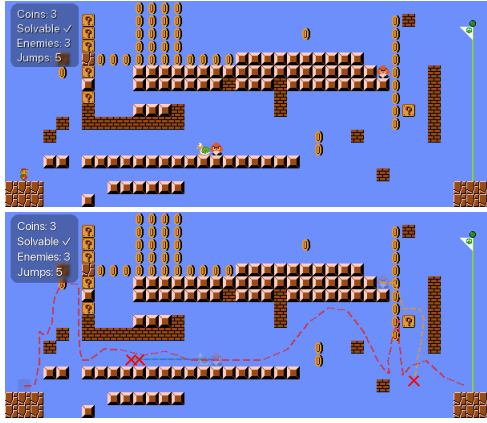


(c) Trial 6

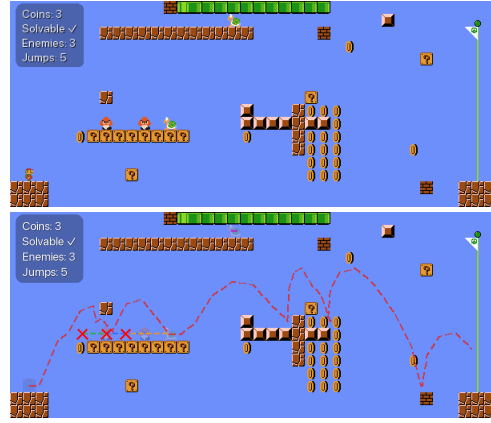


(d) Trial 7

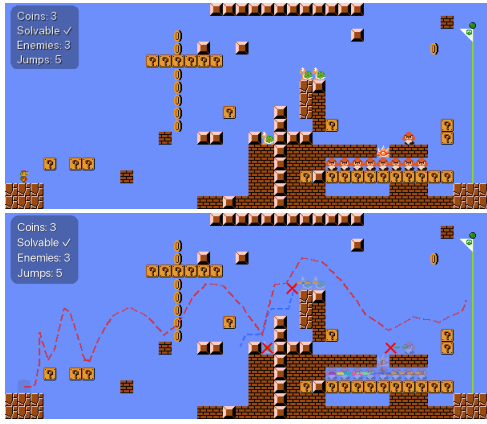
Figure 27: Qualitative examples from the SMB (A*) Evolutionary Tool Search ablation (2/8). Top: level layout. Bottom: agent trajectory. Tags show final gameplay outcomes.



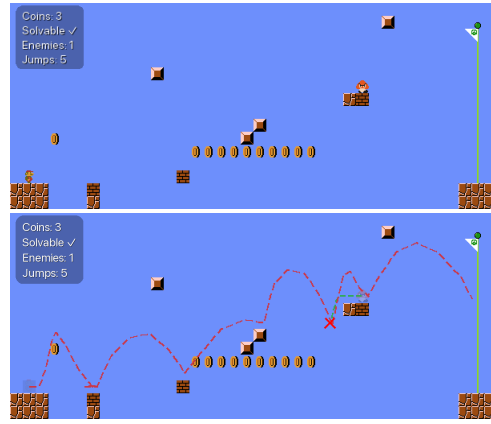
(a) Trial 8



(b) Trial 9

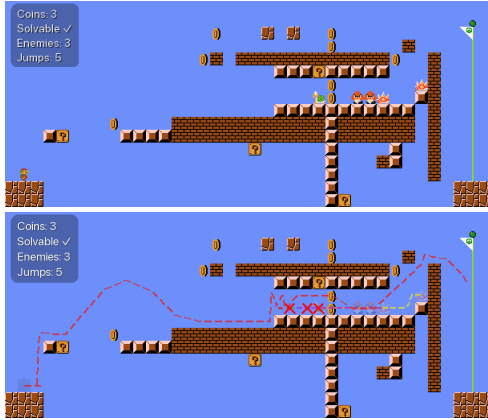


(c) Trial 10

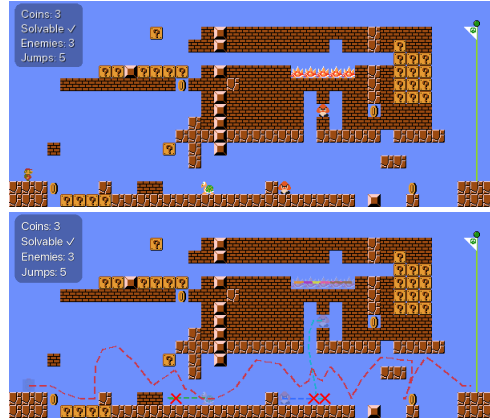


(d) Trial 11

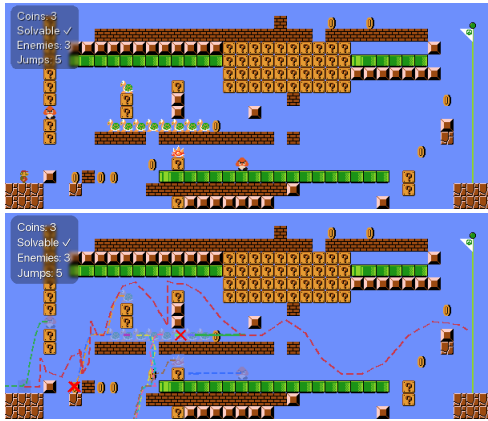
Figure 28: Qualitative examples from the SMB (A*) Evolutionary Tool Search ablation (3/8). Top: level layout. Bottom: agent trajectory. Tags show final gameplay outcomes.



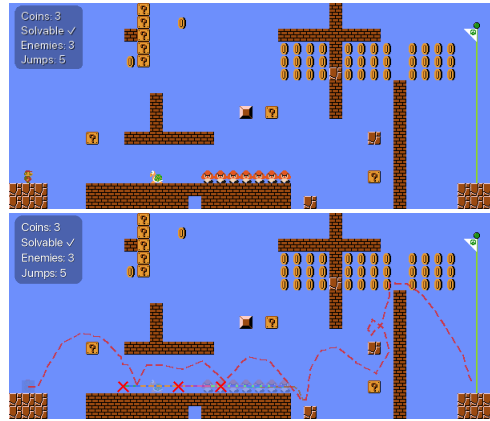
(a) Trial 12



(b) Trial 13



(c) Trial 14

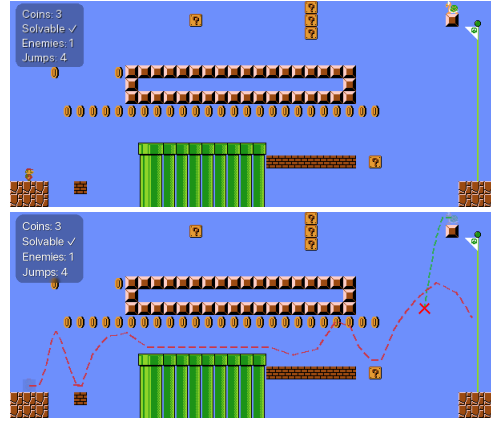


(d) Trial 15

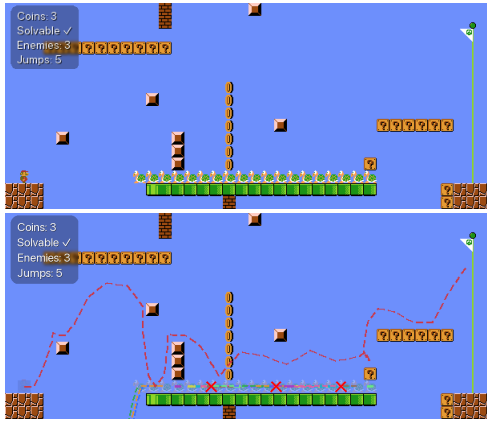
Figure 29: Qualitative examples from the SMB (A*) Evolutionary Tool Search ablation (4/8). Top: level layout. Bottom: agent trajectory. Tags show final gameplay outcomes.



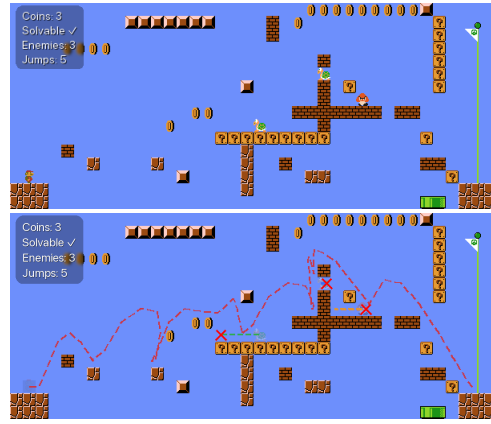
(a) Trial 16



(b) Trial 17

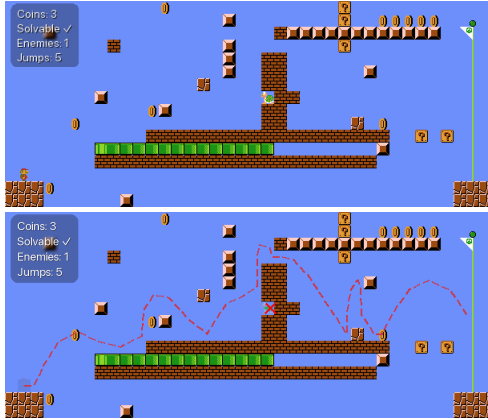


(c) Trial 18

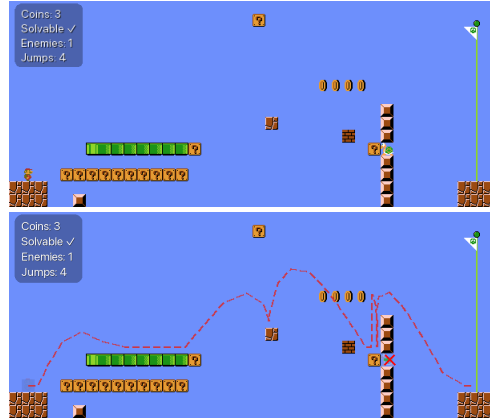


(d) Trial 19

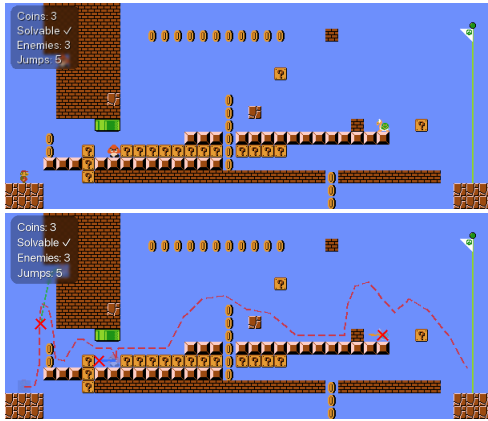
Figure 30: Qualitative examples from the SMB (A*) Evolutionary Tool Search ablation (5/8). Top: level layout. Bottom: agent trajectory. Tags show final gameplay outcomes.



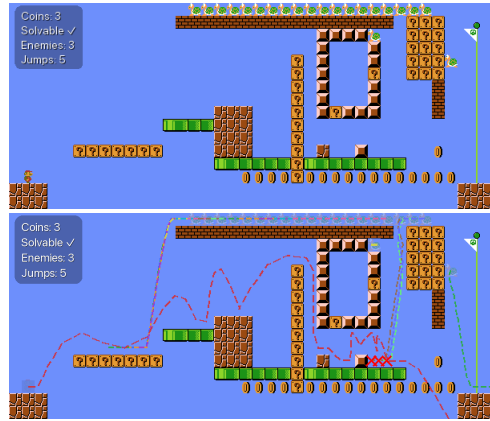
(a) Trial 20



(b) Trial 21

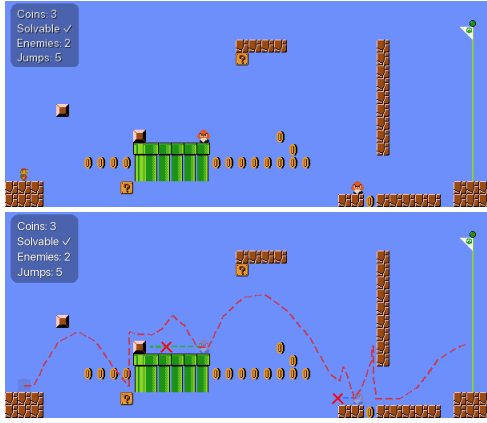


(c) Trial 22

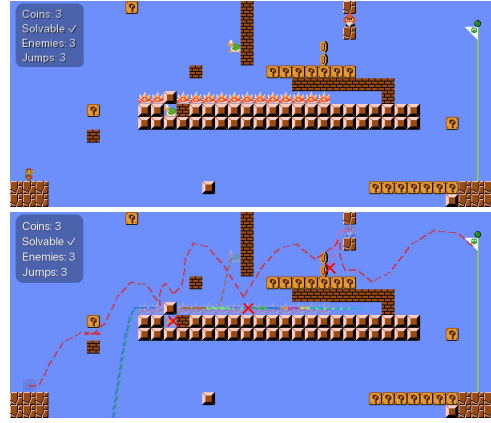


(d) Trial 23

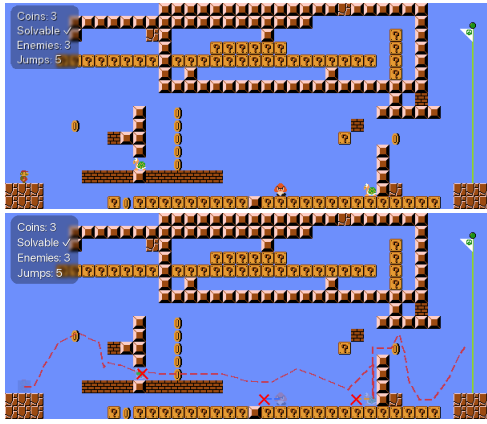
Figure 31: Qualitative examples from the SMB (A*) Evolutionary Tool Search ablation (6/8). Top: level layout. Bottom: agent trajectory. Tags show final gameplay outcomes.



(a) Trial 24



(b) Trial 25

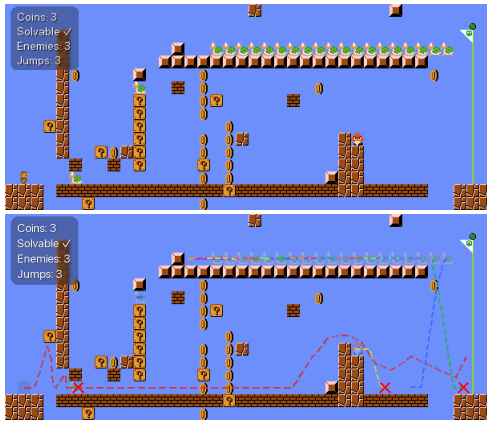


(c) Trial 26

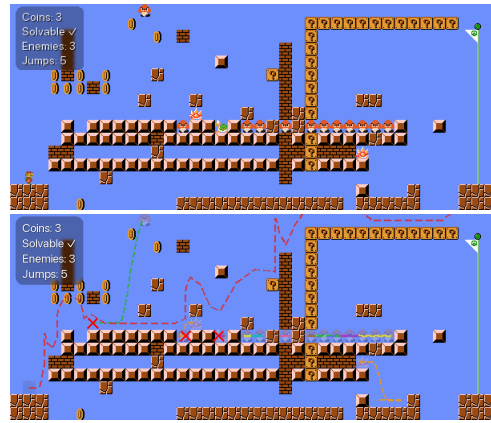


(d) Trial 27

Figure 32: Qualitative examples from the SMB (A*) Evolutionary Tool Search ablation (7/8). Top: level layout. Bottom: agent trajectory. Tags show final gameplay outcomes.



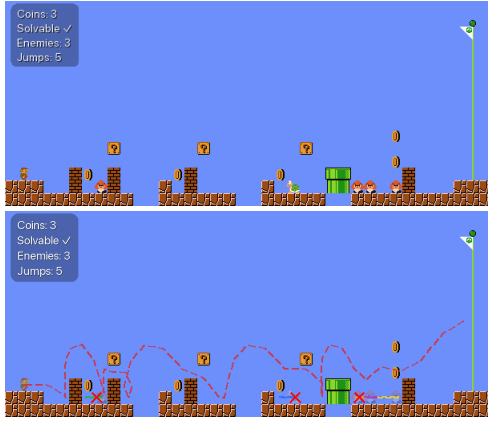
(a) Trial 28



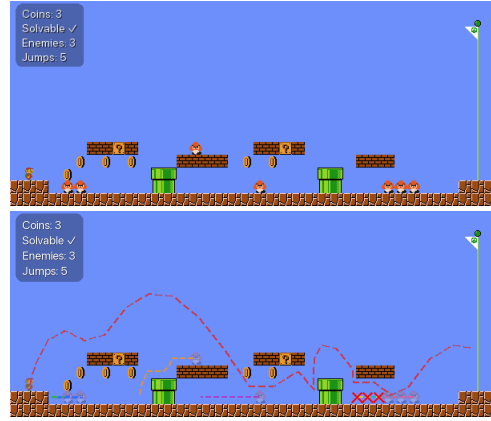
(b) Trial 29

Figure 33: Qualitative examples from the SMB (A*) Evolutionary Tool Search ablation (8/8). Top: level layout. Bottom: agent trajectory. Tags show final gameplay outcomes.

Super Mario Bros (Auto)



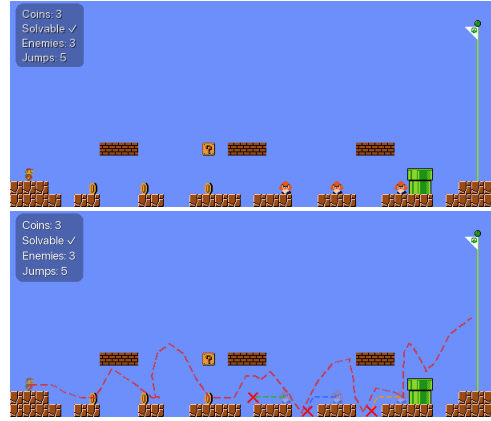
(a) Claude Opus 4.6



(b) Claude Sonnet 4.6

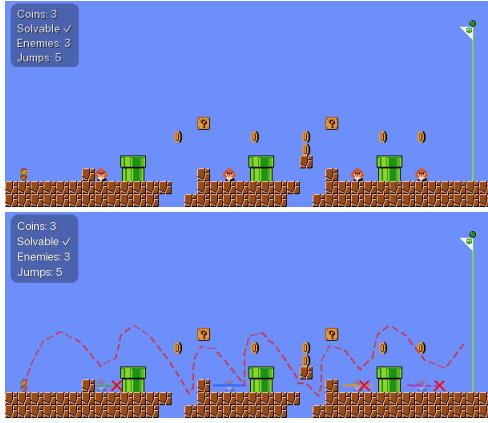


(c) Gemini 2.5 Pro

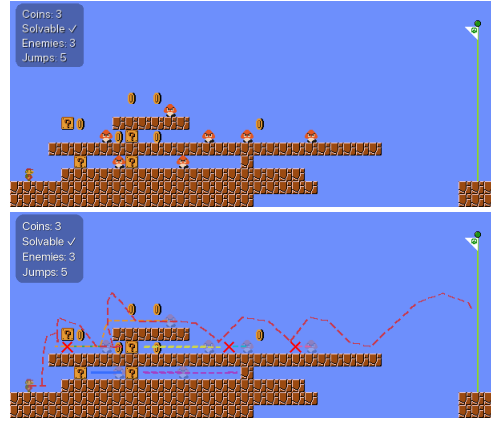


(d) Gemini 3 Flash

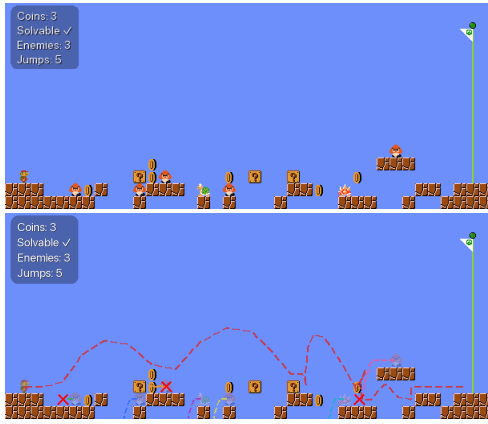
Figure 34: Qualitative comparison of levels generated by different backbone models in the SMB (Auto) problem experiment under the same target metrics (1/2). Top: level layout. Bottom: agent trajectory.



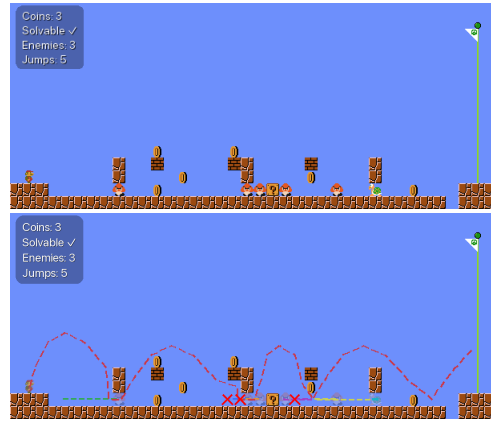
(a) Gemini 3 Pro



(b) GPT-4o mini



(c) o3-mini



(d) Qwen3.5-35B-A3B

Figure 35: Qualitative comparison of levels generated by different backbone models in the SMB (Auto) problem experiment under the same target metrics (2/2).

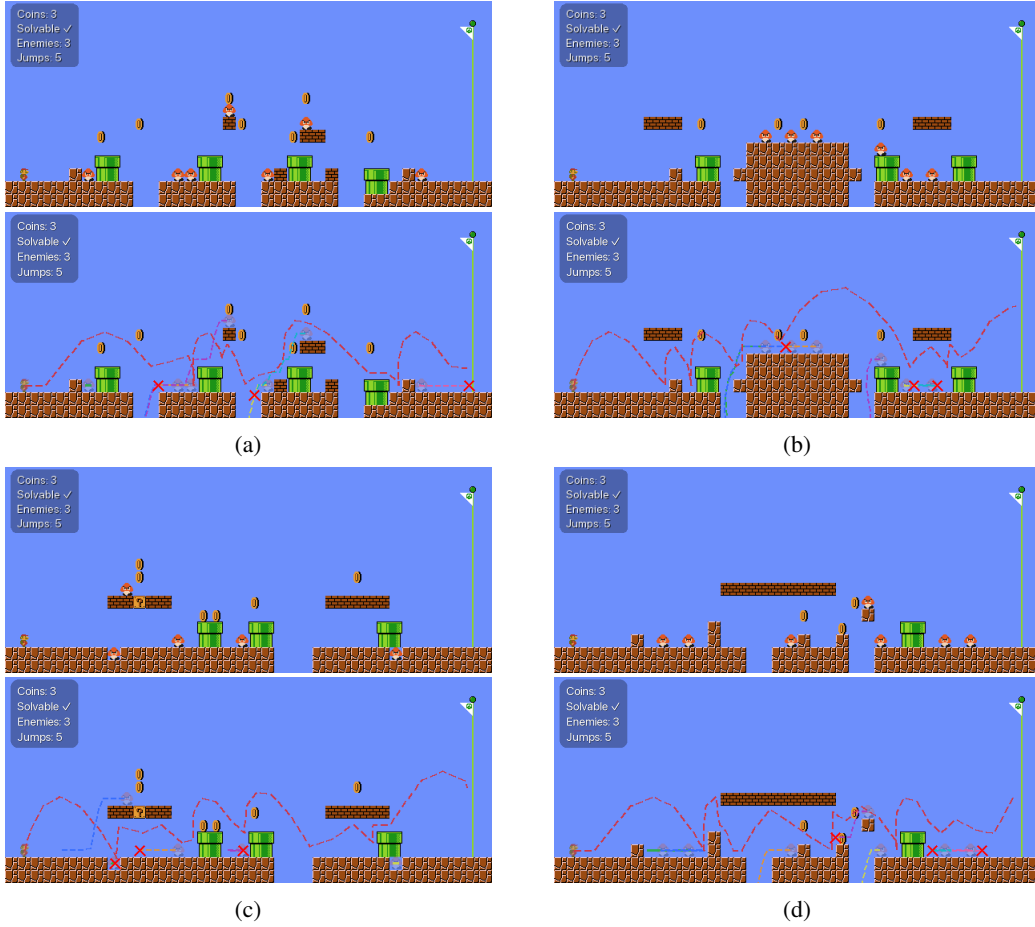


Figure 36: Qualitative diversity of levels generated by Gemini 3 Pro in the SMB (Auto) problem experiment (1/2). Top: level layout. Bottom: agent trajectory.

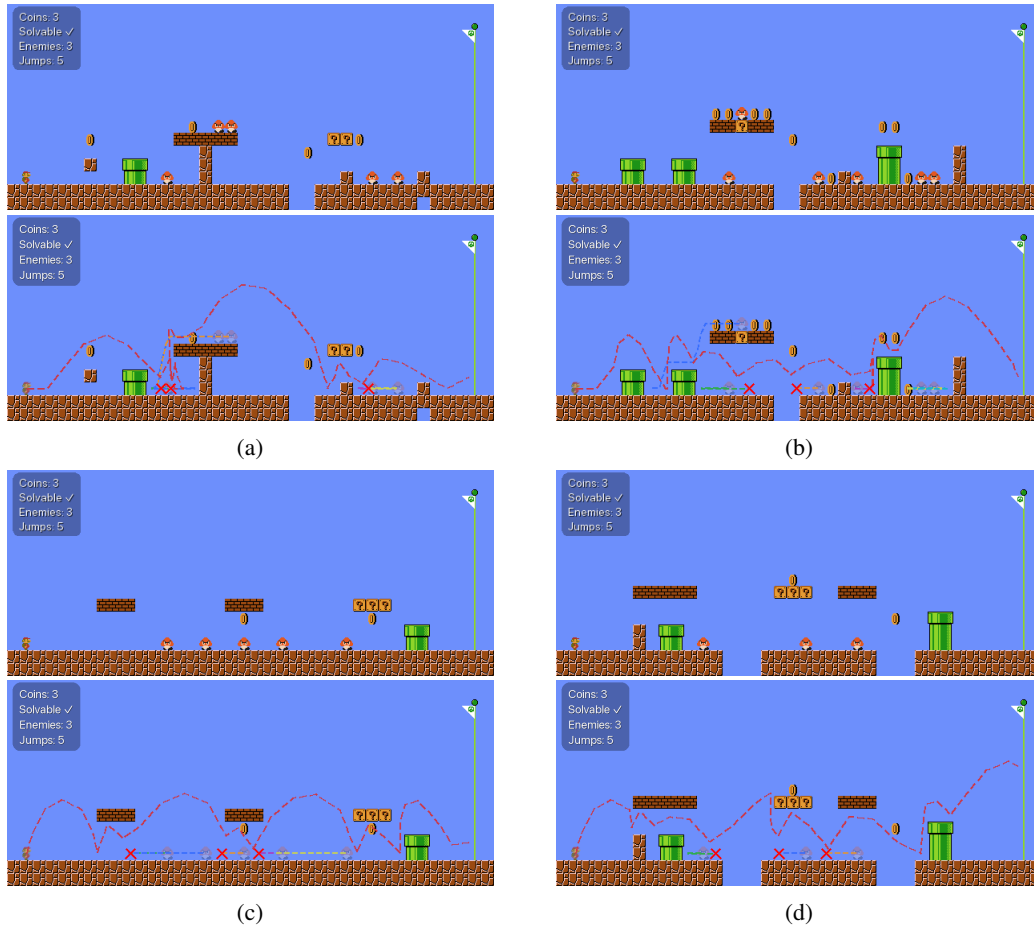


Figure 37: Qualitative diversity of levels generated by Gemini 3 Pro in the SMB (Auto) problem experiment (2/2).

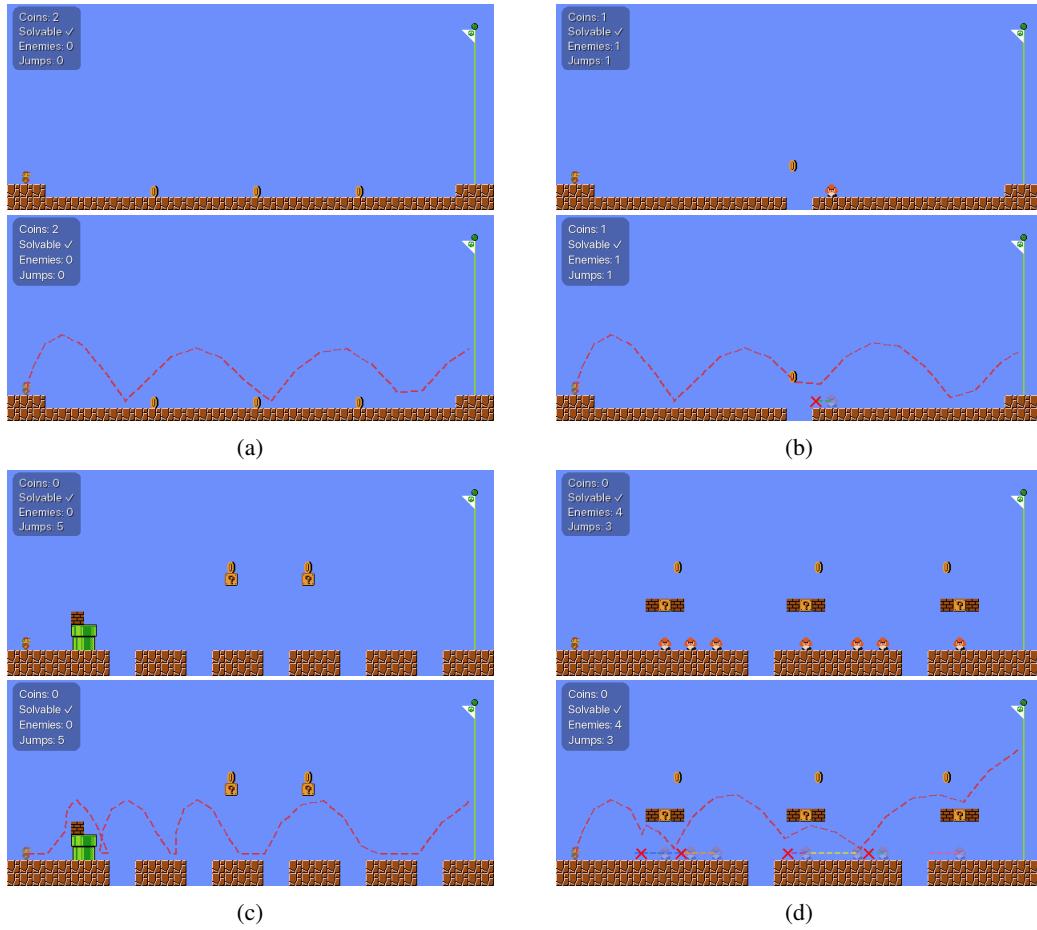


Figure 38: Qualitative controllability examples generated by Gemini 3 Flash in the SMB (Auto) problem experiment under different target metrics (1/2). Top: level layout. Bottom: agent trajectory.

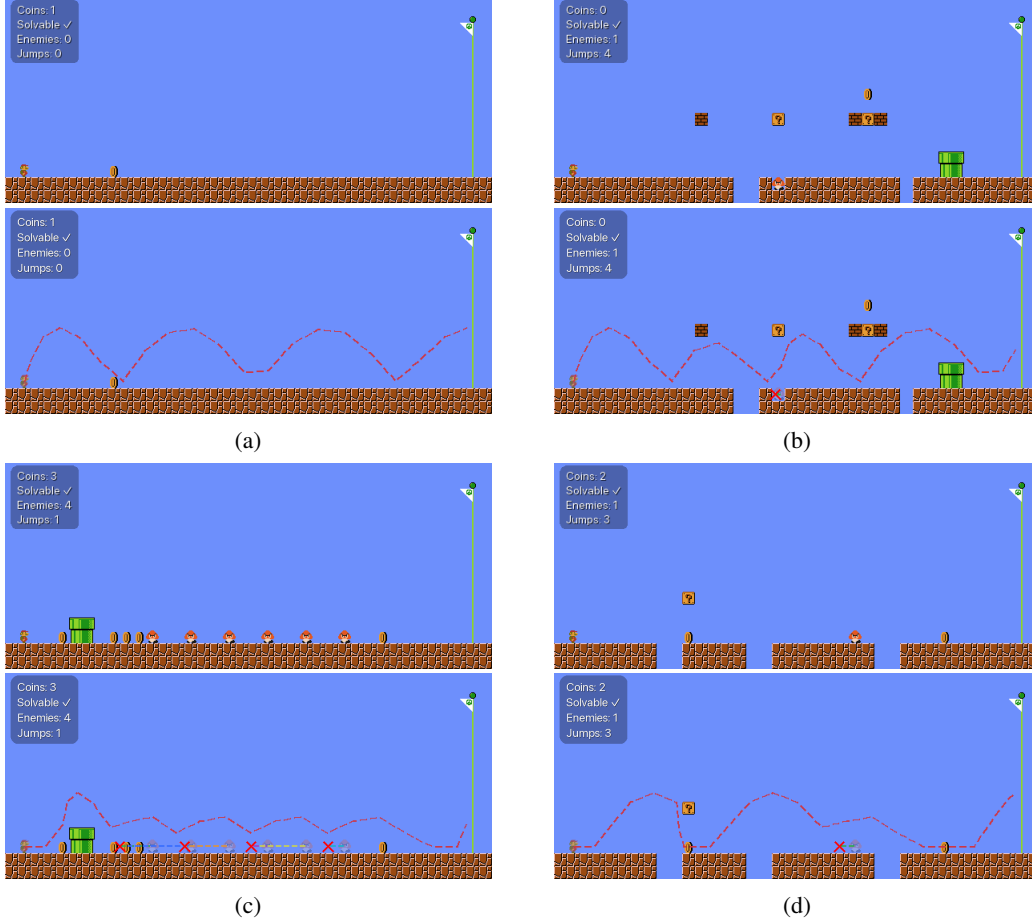


Figure 39: Qualitative controllability examples generated by Gemini 3 Flash in the SMB (Auto) problem experiment under different target metrics (2/2).

K Prompt Details

Although prompt content is problem-specific, all tasks use the same core template: agent role and goals, current level representation, evaluation metrics, score breakdown, termination status, available tools, strategy instructions, and a structured JSON response format. Depending on the configuration, the prompt may also include a tool-call budget and optional extra instructions. For Super Mario Bros., we provide more information in the prompt because the environment is dynamic. Specifically, the prompt further includes an explanation of gameplay-based evaluation and a simulation trajectory map summarizing agent behavior. In multi-turn settings, prompts are additionally prefixed with feedback from the previous step, including accept/reject status, score changes, metric deltas, and tool execution summaries.

Example Prompt From Binary Problem (Step 2)

You are an AI agent optimizing a binary maze level. Your goal is to achieve a longest shortest path of ~256 while maintaining exactly 1 connected region.

Current Level (rows are y, columns are x, 0-indexed):

```

---
#####
## .....#####
## .....#####
### .....#####
## .....#####
#####
#####

```

[illegible]

Description: Ignores the current level and generates a new room-and-corridor layout using Binary Space Partitioning. Recursively splits the space into rectangles, carves a room inside each partition, and connects adjacent rooms with corridors. Works regardless of the current level state. Output has open rectangular rooms linked by narrow passages --- typically 1 connected region with moderate path lengths. Increase splits for more, smaller rooms (longer paths); decrease for fewer, larger rooms (shorter paths). Calling this again discards all previous progress.

Parameters:

- splits (integer) (optional): Number of recursive splits (default 3)
- min_width (integer) (optional): Minimum partition width (default 5)
- min_height (integer) (optional): Minimum partition height (default 5)

generate_ca

Description: REFINEMENT tool: evolves the CURRENT level in-place using cellular automata rules --- does NOT discard previous work. Each iteration, every tile checks its 8 neighbors (Moore neighborhood): if wall-neighbor count <= solid_count the tile becomes wall; if empty-neighbor count >= empty_count the tile becomes empty. Effect: smooths noisy layouts into organic cave-like shapes by removing isolated tiles and filling small holes. IMPORTANT: has no effect on a blank (all-empty or all-wall) level --- needs a mix of wall and empty tiles to work. Best used after generate_random or generate_digger to smooth rough output. Fewer iterations = subtle smoothing; more iterations = stronger smoothing toward large blobs.

Parameters:

- iterations (integer) (optional): Number of CA iterations (default 10)
- solid_count (integer) (optional): Neighbor threshold for becoming/staying wall (default 2)
- empty_count (integer) (optional): Neighbor threshold for becoming empty (default 6)

generate_connect

Description: REFINEMENT tool: post-processes the CURRENT level in-place to fix connectivity --- does NOT discard previous work. Finds all connected empty regions, removes regions smaller than smallest_region_size (fills them with wall), then connects remaining regions with straight corridors. Effect: reduces num_connected_regions toward 1 and increases path length by linking previously isolated areas. IMPORTANT: has no effect on a blank (all-empty) level since it is already one region. Best used as a final pass after any generator (especially generate_random, generate_ca, or generate_digger) to ensure the level is fully connected. Can also be called after manual place_tile edits that may have split the level.

Parameters:

- smallest_region_size (integer) (optional): Minimum region size to keep; smaller regions become wall (default 5)

generate_digger

Description: Ignores the current level and generates a new cave layout using a random-walk digger. Starts from an all-solid grid at a random position and carves empty tiles by walking in random directions, occasionally carving rooms. Stops when the fraction of empty tiles reaches stop_size. Works regardless of the current level state. Output is a single naturally-connected cave with organic, irregular shape --- guaranteed 1 connected region. Higher stop_size = more open space (shorter paths); lower stop_size = tighter tunnels (longer paths). Follow up with generate_ca to smooth rough edges. Calling this again discards all previous progress.

Parameters:

- change_prob (number) (optional): Probability of changing walk direction (default 0.15)
- room_prob (number) (optional): Probability of carving a room instead of a single tile (default 0.01)
- room_size (integer) (optional): Half-size of carved rooms (actual size is 2*room_size+1, default 3)
- stop_size (number) (optional): Stop when empty tile fraction reaches this value (0.0-1.0, default 0.3)

Instructions:

1. Analyze the current level and its metrics
2. Plan edits to improve the score (move metrics toward goals described above)
3. Execute tool calls to modify the level
4. You may call calculate_stats to evaluate changes before committing

Response Format:

Respond with a JSON object of one of these types:

STEP - To execute tools:

```

```json
{
 "type": "STEP",
 "rationale": "explanation of your reasoning",
 "plan": "high-level plan for improvement",
 "tool_calls": [
 {
 "tool_name": "place_wall_segment",
 "parameters": {...}
 },
 {
 "tool_name": "calculate_stats",
 "parameters": {}
 }
],
 "acceptance_hint": "hint about whether to accept changes"
}
```

```

PROPOSE_SKILL - To propose a new tool:

```

```json
{
 "type": "PROPOSE_SKILL",
 "rationale": "why this tool would help",
 "skill_spec": {
 "name": "tool_name",
 "description": "what it does",
 "parameters": {},
 "implementation_hint": "how to implement"
 }
}
...

STOP - To terminate optimization:
```json
{
  "type": "STOP",
  "rationale": "why stopping now",
  "final_notes": "observations about the result"
}
...

```

Respond with ONLY the JSON object, no additional text.

Example Prompt From SMB Problem (Step 61)

You are an AI agent optimizing a Super Mario Bros level. Your goals are:

1. Create a completable level (Mario can reach the end)
2. Have proper tube/pipe structures (tubes must span 2 tiles properly)
3. Achieve ~3 enemies killed, ~5 jumps performed, ~3 coins collected during gameplay simulation
4. Minimize horizontal noise for smoother gameplay
5. Add coins and question blocks for rewards

How Evaluation Works

A Mario AI agent (A* solver) plays your level from left to right. The metrics below reflect the **gameplay outcome**, not the tile layout:

- **enemies_killed**: Number of enemies the player stomped or hit with shells during play. Enemies that fall off cliffs on their own do NOT count. Simply placing enemy tiles does not guarantee kills --- enemies must be on the player's path where they cannot be avoided.
- **coins_collected**: Number of coins the player picked up during play. Coins must be on or near the traversal path.
- **jumps**: Number of voluntary jumps the player performed from the ground. Bouncing off enemies (stomp bounces) does NOT count. More gaps and elevated platforms force more jumps.
- **complete**: Whether the player reached the end flag (1.0 = yes).

To increase **enemies_killed**, place enemies on narrow ground platforms that the player must cross (not on optional platforms). To increase **coins_collected**, place coins along the main path or on mandatory platforms. To increase jumps, create gaps in the ground and elevated platforms that force the player to jump from the ground --- enemy stomps do not count as jumps.

Current Level (rows are y, columns are x, 0-indexed):

```

...
.....
.....
.....
.....
.....
.....GGGG.G.....
.....BBBB.BB.....
.....
.....C.....
.....GCGCGG.....GC...
.....BBB.BB.....BBB.?.
.....X.....
...C.TT.C.....X...C.
.CGCGTT.....X.GGGG.
XXX.XXX.X.....XXXX.X
...

```

Legend: '.' = empty, 'X' = solid floor, 'L' = ladder, 'B' = brick, '?' = question block, 'T' = tube, 'C' = coin, 'G' = goomba, 'K' = koopa, 'Y' = spiny

Simulation Trajectory Map (last evaluation)

```

...
.....
.....***.....
.....**..*.....
.....**..*.....
.....**..*.....

```

```

.....*.....
.....*!..GGGG.G*.....
.....**..BBB.BB*.....
.....*.....*.....
.....*.....*.....C.....*.....
...*...GCGCGG.....GC...
..*...BBB.BB.....BBB.?.
**.....X.....
**..C.TT..C.....X...C.
.!GCGTT.....X.GGGG.
XXX.XXX.X.....XXX.X
```


Legend: '*' = Mario's path (empty tiles only), '!' = enemy killed here

Note: Enemies move during simulation. '!' marks where Mario was when the kill happened, not the enemy's spawn position. Enemies that were avoided or not reached are still shown at their original tile positions (G/K/Y).

Current Level Metrics:

- complete: 100.0% (goal: 100%)
- enemies_killed: 2 (goal: ~3)
- coins_collected: 3 (goal: ~3)
- jumps: 4 (goal: ~5) [stomp bounces: 2, not counted]
- tube_issues: 0 (goal: 0)
- empty_ratio: 88.5%
- noise: 0.363 (goal: minimize)
- simulation: ran
- playable: yes

Simulation Results

The Mario AI agent played through the level. Here is what happened:

- Completion: 100% (reached the flag)
- Enemies killed: 2 (goomba stomped at tile [1, 14], goomba stomped at tile [17, 6])
- Coins collected: 3
- Jumps performed: 4 (stomp bounces: 2, not counted as jumps)
- Mario traversed from column 0 to column 31

Enemy behavior during simulation:

- Goomba (G): Walks horizontally, reverses on walls. Speed ~1.75 px/frame.
- Koopa (K): Green koopa walks horizontally, falls off cliffs. Becomes a kickable shell when stomped.
- Spiny (Y): Walks horizontally like goomba but CANNOT be stomped (hurts Mario). Must be avoided or killed with shell/fireball.

Score: 70.00 (complete=100%, tubes=0, enemies_killed=2, coins_collected=3, jumps=4)

Termination Status:

Changes: 89/1536 (5.8% used, 1447 remaining)

Available Tools:

place_tile

Description: Places a tile on the level. Supported tile types: empty, solid, ladder, brick, question, tube, coin, goomba, koopa, spiny. Supports three modes:

- 'single': place one tile at (y, x).
- 'line': place a straight horizontal or vertical segment. Specify the endpoint using ONE of: (a) direction ('up'/'down'/'left'/'right') + length, (b) end_y + end_x for an explicit endpoint, (c) end_x only (end_y defaults to y, drawing a horizontal line), or (d) end_y only (end_x defaults to x, drawing a vertical line). Example horizontal line: {mode:'line', y:5, x:0, end_x:10} Example vertical line: {mode:'line', y:0, x:3, end_y:8}
- 'rect': fill a rectangle from (y, x) to (end_y, end_x). Requires end_y and end_x.

Parameters:

- mode (string) (required): Mode of operation: 'single' for one tile, 'line' for a segment, 'rect' for rectangle
- tile_type (string) (required): Type of tile to place: empty, solid, ladder, brick, question, tube, coin, goomba, koopa, spiny
- y (integer) (required): Row coordinate (0-indexed). For line/rect, this is start_y.
- x (integer) (required): Column coordinate (0-indexed). For line/rect, this is start_x.
- end_y (integer) (optional): Ending row coordinate. Required for rect mode. For line mode: if only end_y is given (no end_x), draws a vertical line from y to end_y at column x. If both end_y and end_x are given, draws a line from (y,x) to (end_y,end_x).
- end_x (integer) (optional): Ending column coordinate. Required for rect mode. For line mode: if only end_x is given (no end_y), draws a horizontal line from x to end_x at row y. If both end_y and end_x are given, draws a line from (y,x) to (end_y,end_x).
- direction (string) (optional): Direction to extend from (y,x) --- alternative to end_y/end_x for line mode. Must be paired with 'length'.
- length (integer) (optional): Number of tiles to place in 'direction' (for line mode with direction). Must be paired with 'direction'.
- filled (boolean) (optional): For rect mode: if true, fill entire rectangle; if false, only draw border

Instructions:


```

1. Ensure the level is completable (Mario can traverse from start to end)
2. Place solid ground (X) for Mario to walk on
3. Add platforms using bricks (B) and question blocks (?)
4. Place enemies (G, K, Y) on solid ground along the player's path --- enemies the player can avoid will not count
5. Place coins (C) along the main traversal path --- coins the player never reaches will not count
6. Avoid creating impossible jumps or dead ends

## Response Format:

Respond with a JSON object of one of these types:

### STEP - To execute tools:

```json

```
{
  "type": "STEP",
  "rationale": "explanation of your reasoning",
  "plan": "high-level plan for improvement",
  "tool_calls": [
    {"tool_name": "place_tile", "parameters": {"mode": "single", "tile_type": "solid", "y": 15,
      "x": 1}}
  ],
  "acceptance_hint": "hint about whether to accept changes"
}
```

PROPOSE_SKILL - To propose a new tool:

```json

```
{
 "type": "PROPOSE_SKILL",
 "rationale": "why this tool would help",
 "skill_spec": {
 "name": "tool_name",
 "description": "what it does",
 "parameters": {},
 "implementation_hint": "how to implement"
 }
}
```

### STOP - To terminate optimization:

```json

```
{
  "type": "STOP",
  "rationale": "why stopping now",
  "final_notes": "observations about the result"
}
```

Respond with ONLY the JSON object, no additional text.

Extra Instruction:

Make level with two elevated platforms, requires travelling to upper platform to solve

Example Feedback Prefix to LLM Agent From SMB Problem (Step 61)

[Previous step result: REJECTED - rejected (score: 70.00 -> 50.00)]

Metrics change from your last edit:

enemies_killed: 2 → 3 (target: ~3)

coins_collected: 3 → 1 (target: ~3)

jumps: 4 → 3 (target: ~5)

complete: 100.0% → 100.0% (target: ~1.0)

tube_issues: 0 → 0

noise: 0.363 → 0.178

Tool results: 16 calls, 16 succeeded, 102 tiles changed